

TITLE OF THE PROJECT REPORT

**LLM LAWYER: A RAG-BASED LEGAL
REASONING AND ASSISTANCE SYSTEM USING
LLMS**

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Vulli Mohan Durga Prakash (22BCE9657)

Alur Manideep (22BCE20036)

Under the guidance of:

Dr. Jonnadula Harikiran

Associate Professor

School of Computer Science and Engineering

VIT-AP University, Amaravati



**VIT-AP
UNIVERSITY**

Name of the school

SCHOOL OF Computer Science and Engineering

DECLARATION

We hereby declare that the thesis entitled "LLM LAWYER: A RAG-BASED LEGAL REASONING AND ASSISTANCE SYSTEM USING LLMS" submitted by us, for the award of the degree of Bachelor of Technology in Computer Science and Engineering at VIT-AP University is a record of bonafide work carried out by us under the supervision of Dr. Jonnadula Harikiran, Associate Professor.

We further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Amaravati

Date: 01-05-2026

Vulli Mohan Durga Prakash
22BCE9657

Alur Manideep
22BCE20036

CERTIFICATE

This is to certify that the Senior Design Project titled "LLM LAWYER: A RAG-BASED LEGAL REASONING AND ASSISTANCE SYSTEM USING LLMS" that is being submitted by Vulli Mohan Durga Prakash (22BCE9657) and Alur Manideep (22BCE20036) is in partial fulfillment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering, and is a record of bonafide work done under my guidance. The contents of this project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Dr. Jonnadula Harikiran

Associate Professor

School of CSE, VIT-AP University

The thesis is Satisfactory / Unsatisfactory

Internal Examiner

External Examiner

Approved by:

PROGRAM CHAIR

B.Tech. CSE

DEAN

School of Computer Science and Engineering

ABSTRACT

The rapid advancement of Large Language Models (LLMs) has enabled intelligent systems capable of understanding and generating human-like text. However, in domains such as law, accuracy, traceability, and reliability are absolutely critical. Standard LLM-based systems often suffer from hallucinations, lack of source attribution, and an inability to perform structured legal reasoning. These limitations make them unsuitable for deployment in real-world legal environments where every claim must be verifiable and traceable to an authenticated source.

This project, AI Lawyer Pro, was developed to address these fundamental limitations by building a Retrieval-Augmented Generation (RAG) system specifically tailored for legal document understanding. The system was designed to analyze First Information Reports (FIRs) and legal documents based on the updated Indian Criminal Laws—the Bharatiya Nyaya Sanhita (BNS), Bharatiya Nagarik Suraksha Sanhita (BNSS), and Bharatiya Sakshya Adhiniyam (BSA)—which replaced the outdated IPC and CrPC frameworks.

The system evolved through multiple development phases. It began with basic chunk-based retrieval from Official Bare Act PDFs, then incorporated semantic search using SentenceTransformers and ChromaDB, followed by a Cross-Encoder reranking layer for improved precision. A critical hallucination prevention system using Regex-based citation verification was implemented to ensure all legal references are validated against the actual database. The architecture was further upgraded to a Hybrid RAG approach combining Vector Search (semantic) with GraphRAG (relational) using NetworkX, featuring over 1,140 nodes and 1,860 edges encoding PART_OF_CHAPTER and REFERENCES relationships.

Advanced prompt engineering techniques including zero-temperature inference, relevancy filtering, and strict JSON schema enforcement using few-shot prompting were applied to eliminate unstructured or fabricated outputs. The frontend, developed with Streamlit, features a chat-based interface with authentication, session history, and a dynamic visualization dashboard powered by Plotly, showing

case timelines, risk scores, and section impact charts. A dual-pipeline OCR system using Tesseract was integrated to handle scanned legal PDFs.

The result is a robust, explainable, and scalable Legal AI system that combines the semantic power of vector embeddings with the structural intelligence of knowledge graphs, delivering high-precision, verifiable, and human-interpretable answers to complex legal queries.

Keywords: Legal AI, Retrieval-Augmented Generation, GraphRAG, Hybrid RAG, Hallucination Prevention, BNS, BNSS, BSA, ChromaDB, LLM, Prompt Engineering, OCR, Streamlit, FastAPI

ACKNOWLEDGEMENT

It is our pleasure to express our deep sense of gratitude to Dr. Jonnadula Harikiran, Associate Professor, School of Computer Science and Engineering, VIT-AP University, for his constant guidance, continual encouragement, and understanding throughout the duration of this project. His expertise in Artificial Intelligence and Natural Language Processing has been an invaluable asset in shaping the direction and quality of this work.

We would like to express our gratitude to the Chancellor, Vice Presidents, Vice Chancellor, and the Dean of the School of Computer Science and Engineering for providing an inspiring environment and for their encouragement during the tenure of the course.

We express our heartfelt thanks to the Program Chair and all teaching staff and members working as pillars of our university for their selfless enthusiasm coupled with timely encouragement, which prompted the acquisition of the requisite knowledge to finalize this course study successfully.

We gratefully acknowledge the contributors and maintainers of the open-source tools and frameworks that were central to this work, including the developers of PyPDF2, SentenceTransformers, ChromaDB, NetworkX, Groq API, Ollama, FastAPI, Streamlit, Plotly, and Tesseract OCR. We also acknowledge the Government of India for making the Bare Act legal texts of BNS, BNSS, and BSA publicly available, which served as the ground-truth knowledge base for this system.

We would like to thank our parents for their constant moral support and encouragement. It is a pleasure to thank our friends who persuaded and encouraged us to take up and complete this task. Finally, we express our gratitude and appreciation to all those who have helped us directly or indirectly toward the successful completion of this project.

Place: Amaravati

Date: : 01-05-2026

Vulli Mohan Durga Prakash

Alur Manideep

TABLE OF CONTENTS

DECLARATION	2
CERTIFICATE	3
ABSTRACT	4
ACKNOWLEDGEMENT	6
TABLE OF CONTENTS	7
LIST OF FIGURES	12
LIST OF TABLES	13
LIST OF ABBREVIATIONS	14
Chapter 1: Introduction	15
1.1 Background and Motivation	15
1.2 Problem Statement	16
1.3 Research Objectives	16
1.4 Project Contributions	17
1.5 Scope of the Study	17
1.6 Organization of the Report	17
Chapter 2: Literature Review	19
2.1 Retrieval-Augmented Generation (RAG)	19
2.2 Vector Databases and Semantic Search	19
2.3 Graph-Based RAG (GraphRAG)	20
2.4 Hallucination in Legal AI	20
2.5 Hybrid Retrieval Systems	20
2.6 Gaps in Existing Literature	21
Chapter 3: System Architecture	22
3.1 Architecture Overview	22
3.2 Technology Stack	24
3.3 Data Flow	25
Chapter 4: Data Engineering and Preprocessing	27
4.1 Overview	27
4.2 Data Source Selection – A Critical Decision	27
4.3 Text Extraction	27
4.4 Section Segmentation	28
4.5 Metadata Enrichment	28

4.6 Chunking Strategy.....	28
Chapter 5: Semantic Representation and Vector Database.....	30
5.1 Embedding Model.....	30
5.2 ChromaDB Setup	30
5.3 Database Schema.....	31
5.4 Why Semantic Search Over Keyword Search.....	31
Chapter 6: Basic RAG Pipeline	32
6.1 Pipeline Overview	32
6.2 Retrieval Configuration.....	32
6.3 Limitations Identified.....	32
Chapter 7: Reranking Mechanism	34
7.1 The Problem with Bi-Encoder Retrieval.....	34
7.2 Cross-Encoder Reranking Implementation	34
7.3 Impact of Reranking.....	35
Chapter 8: Hallucination Prevention System.....	37
8.1 The Hallucination Problem in Legal AI.....	37
8.2 Citation Verification System.....	37
8.3 Engineering Significance	38
Chapter 9: GraphRAG – Graph-Based Retrieval.....	39
9.1 Motivation for Graph-Based Retrieval.....	39
9.2 Legal Knowledge Graph Construction.....	39
9.2.1 Node Creation.....	39
9.2.2 Edge Creation	40
9.2.3 Graph Statistics.....	40
9.3 Graph Traversal for Query Answering.....	41
Chapter 10: Hybrid RAG Architecture	43
10.1 Combining Vector and Graph Retrieval.....	43
10.2 Comparison: Traditional RAG vs Hybrid RAG.....	44
10.3 Engineering Rationale	44
Chapter 11: Advanced Prompt Engineering	46
11.1 The Importance of Prompt Engineering in Legal AI	46
11.2 Zero-Temperature Inference	46
11.3 Relevancy Filtering	46
11.4 Structured Output Schema Enforcement.....	47

11.5 Error Handling for Schema Drift	48
Chapter 12: Frontend Development and Visualization.....	49
12.1 Streamlit Frontend.....	49
12.2 Authentication System	49
12.3 Chat Interface	49
12.4 Custom CSS and Layout	50
12.5 Plotly Visualization Dashboard.....	50
12.5.1 Case Timeline – Scatter Plot	50
12.5.2 Case Risk Score – Gauge Chart.....	51
12.5.3 Section Impact – Horizontal Bar Chart	52
Chapter 13: OCR Integration	54
13.1 The Scanned PDF Problem	54
13.2 Dual-Pipeline Architecture.....	54
13.3 Tesseract Configuration	55
13.4 Impact on System Utility.....	55
Chapter 14: Deployment Strategy.....	57
14.1 Environment Isolation	57
14.2 Multi-Service Execution	57
14.3 Ngrok Tunneling for Live Demo	57
14.4 Environment Variables.....	58
Chapter 15: Legal Framework Transition – IPC/CrPC to BNS/BNSS/BSA.....	59
15.1 The Legal Reform Context.....	59
15.2 Why This Transition Mattered	59
15.3 System Adaptability Demonstrated.....	60
Chapter 16: Experimental Phase – Page-Based Retrieval	61
16.1 Experiment Motivation	61
16.2 Page-Based Approach	61
16.3 Limitations Discovered	61
16.4 Decision.....	62
Chapter 17: Results and Evaluation.....	63
17.1 Evaluation Methodology	63
17.2 Retrieval Precision Improvement.....	63
17.3 Hallucination Rate.....	63
17.4 System Performance.....	64

17.5 Qualitative Assessment	64
Chapter 18: Future Enhancements	65
18.1 Hybrid Search (BM25 + Semantic).....	65
18.2 Enhanced GraphRAG with Neo4j.....	65
18.3 Judicial Precedent Integration	65
18.4 Multi-Language Support	65
18.5 Fine-Tuned Legal LLM.....	66
18.6 UI Enhancements	66
Chapter 19: Conclusion.....	67
References	69
Appendix A: System Implementation Details	70
A.1 Data Engineering Pipeline – Detailed Implementation	70
A.1.1 PDF Text Extraction and Cleaning	70
A.1.2 Metadata Enrichment Process.....	70
A.2 ChromaDB Collection Structure	71
A.3 Cross-Encoder Reranking – Technical Details	71
A.4 GraphRAG Implementation – NetworkX Details	72
Appendix B: Testing and Sample Query Analysis	73
B.1 Test Query Categories	73
B.2 Sample Query and Response Analysis	73
B.2.1 Sample Query 1: Offense Classification.....	73
B.2.2 Sample Query 2: Multi-Hop Procedural Query	74
B.3 Edge Cases Tested	74
Appendix C: Technology Comparison and Selection Rationale	76
C.1 Embedding Model Comparison.....	76
C.2 Vector Database Comparison	76
C.3 LLM Provider Comparison	76
C.4 Graph Library Comparison.....	77
Appendix D: Project Evolution Timeline	78
D.1 Development Milestones	78
D.2 Key Architectural Decisions and Their Impact	79
Appendix E: System Requirements and Setup	81
E.1 Hardware Requirements	81
E.2 Software Dependencies.....	81

E.3 Setup Instructions	82
E.4 Project Directory Structure	83
E.5 API Endpoints.....	84
E.6 Security Considerations	84
Appendix F: Comparison with Related Work	85
F.1 AI Lawyer Pro vs. Existing Legal AI Systems	85
F.2 Key Differentiators	85

LIST OF FIGURES

Figure	Caption
Figure 3.1	System Architecture – Microservices Overview
Figure 3.2	Data Engineering Pipeline – PDF to Vector DB
Figure 4.1	Chunking Strategy – 1000 char / 200 overlap
Figure 5.1	Vector Embedding Flow – SentenceTransformers to ChromaDB
Figure 6.1	Basic RAG Pipeline Flow
Figure 7.1	Reranking Mechanism – Cross-Encoder Integration
Figure 8.1	Hallucination Prevention System – Citation Verification
Figure 9.1	GraphRAG Node-Edge Structure (1140 Nodes, 1860 Edges)
Figure 9.2	Legal Graph Relationships – PART_OF_CHAPTER & REFERENCES
Figure 10.1	Hybrid RAG Architecture – Vector + Graph Fusion
Figure 11.1	Prompt Engineering Schema – Few-Shot JSON Output
Figure 12.1	Streamlit UI – Chat Interface with Sources Tab
Figure 12.2	Plotly Visualization – Case Timeline Scatter Plot
Figure 12.3	Plotly Visualization – Risk Score Gauge Chart
Figure 12.4	Plotly Visualization – Section Impact Bar Chart
Figure 13.1	OCR Dual-Pipeline – Text Extraction + Tesseract Fallback
Figure 14.1	Deployment Architecture – Uvicorn + Streamlit + Ngrok

LIST OF TABLES

Table	Caption
Table 3.1	Technology Stack Overview
Table 4.1	Chunking Strategy – Parameters & Justification
Table 5.1	ChromaDB Metadata Schema
Table 7.1	Reranking Model – Performance Comparison
Table 9.1	GraphRAG – Node and Edge Statistics
Table 10.1	Comparison: Traditional RAG vs Hybrid RAG
Table 14.1	Deployment Configuration – Ports and Services
Table 15.1	Legal Framework Comparison – IPC/CrPC vs BNS/BNSS/BSA

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
GraphRAG	Graph-based Retrieval-Augmented Generation
NLP	Natural Language Processing
BNS	Bharatiya Nyaya Sanhita
BNSS	Bharatiya Nagarik Suraksha Sanhita
BSA	Bharatiya Sakshya Adhiniyam
IPC	Indian Penal Code
CrPC	Code of Criminal Procedure
FIR	First Information Report
API	Application Programming Interface
REST	Representational State Transfer
UI	User Interface
OCR	Optical Character Recognition
JSON	JavaScript Object Notation
PDF	Portable Document Format
DB	Database
VIT-AP	Vellore Institute of Technology – Andhra Pradesh
CSE	Computer Science and Engineering

Chapter 1: Introduction

1.1 BACKGROUND AND MOTIVATION

The Indian legal system, one of the most complex and voluminous in the world, underwent a landmark transformation in 2023 with the introduction of three new criminal laws: the Bharatiya Nyaya Sanhita (BNS), Bharatiya Nagarik Suraksha Sanhita (BNSS), and Bharatiya Sakshya Adhiniyam (BSA). These laws replaced the centuries-old Indian Penal Code (IPC), Code of Criminal Procedure (CrPC), and Indian Evidence Act respectively, introducing updated definitions, restructured sections, and modernized procedural guidelines. This transition created an urgent need for intelligent systems capable of interpreting and explaining these new legal frameworks.

The widespread adoption of Large Language Models (LLMs) in recent years has opened new possibilities for automated legal assistance. However, standard LLM applications suffer from a fundamental problem in the legal domain: hallucination. LLMs, when queried about specific legal provisions, often generate plausible-sounding but incorrect section numbers, fabricated case references, and non-existent legal definitions. In a domain where accuracy and source traceability are non-negotiable, such behavior renders standard LLMs dangerous and unreliable.

Retrieval-Augmented Generation (RAG) addresses this challenge by grounding LLM responses in verified, retrieved content from a curated knowledge base. Rather than relying purely on parametric knowledge baked into model weights during training, RAG systems retrieve the most relevant context from an external database and pass it to the LLM as part of the prompt. This dramatically reduces hallucination and enables source attribution. The AI Lawyer Pro project applies this RAG paradigm to the Indian legal domain, building a system that can accurately analyze legal documents, retrieve applicable laws, and generate structured, verifiable responses.

1.2 PROBLEM STATEMENT

The primary objective of this project is to design and implement an AI system capable of analyzing legal documents—particularly First Information Reports (FIRs)—and answering queries with high accuracy, contextual relevance, and verifiable sources, based on the updated Indian criminal law framework (BNS, BNSS, BSA).

The key challenges identified include:

- Legal datasets are complex and unstructured, with sections that do not cleanly separate in raw text extraction
- Existing CSV-based datasets lacked critical metadata such as bailable/non-bailable classification and procedural details
- LLMs tend to generate incorrect legal references and non-verifiable answers without proper grounding
- Standard vector search lacks relational understanding needed for multi-step legal reasoning
- Some legal PDFs exist as scanned images, requiring OCR capability for text extraction

1.3 RESEARCH OBJECTIVES

1. Build a reliable data pipeline from Official Bare Act PDFs with accurate section segmentation and metadata enrichment
2. Implement semantic retrieval using vector embeddings to move beyond keyword-based search
3. Design a reranking mechanism to improve the precision of retrieved legal content
4. Develop a robust hallucination prevention system using citation verification against the database
5. Upgrade the retrieval architecture to a Hybrid RAG combining vector search with graph-based relational reasoning
6. Apply advanced prompt engineering techniques to ensure structured, deterministic LLM outputs
7. Build an interactive frontend with visualizations for case timelines, risk scores, and section impact
8. Transition the system from IPC/CrPC to BNS/BNSS/BSA to maintain real-world legal relevance

1.4 PROJECT CONTRIBUTIONS

The key contributions of this project are:

- A complete end-to-end Legal AI pipeline from raw PDF ingestion to structured response generation
- A Hybrid RAG architecture combining semantic vector search with GraphRAG using NetworkX (1,140+ nodes, 1,860+ edges)
- A citation verification system that cross-checks LLM outputs against ChromaDB metadata to flag hallucinations
- A zero-temperature, schema-enforced prompt engineering strategy ensuring deterministic legal outputs
- A dual-pipeline OCR system for handling both digital and scanned PDF legal documents
- An interactive Plotly-powered visualization dashboard for legal case insights
- A microservices architecture (Streamlit + FastAPI) enabling scalable, industry-grade deployment

1.5 SCOPE OF THE STUDY

This project focuses on the analysis of Indian criminal law documents under the BNS, BNSS, and BSA frameworks. The system is designed to process uploaded FIR documents, retrieve applicable legal sections, and generate structured analytical outputs. The scope includes offline testing with Ngrok-based public tunneling for demonstration. Future expansions may include civil law, constitutional law, and real-time legal database integration.

1.6 ORGANIZATION OF THE REPORT

The report is organized as follows: Chapter 2 presents the Literature Review. Chapter 3 describes the System Architecture. Chapters 4 through 9 detail the step-by-step development phases. Chapter 10 covers the Hybrid RAG upgrade. Chapter 11 discusses Prompt Engineering. Chapter 12 describes the Frontend and Visualization. Chapter 13 covers OCR Integration. Chapter 14 details the Deployment Strategy. Chapter 15 presents the Legal Framework Transition. Chapter 16 discusses Results and Evaluation. Chapter 17

covers Future Enhancements. Chapter 18 concludes the report. References follow the conclusion.

Chapter 2: Literature Review

2.1 RETRIEVAL-AUGMENTED GENERATION (RAG)

Lewis et al. (2020) introduced Retrieval-Augmented Generation as a paradigm for knowledge-intensive NLP tasks. Their work demonstrated that combining a dense retrieval system with a sequence-to-sequence generator significantly outperforms purely parametric models on open-domain question answering benchmarks. The core insight is that while LLMs encode vast general knowledge in their parameters, this knowledge is static and may be outdated, incomplete, or incorrect for specialized domains. RAG addresses this by dynamically retrieving relevant context from an external corpus at inference time.

In the legal domain, this paradigm is particularly powerful. Legal knowledge is highly domain-specific, frequently updated, and demands precise source attribution. RAG systems can be grounded in curated legal corpora, ensuring that every generated claim is traceable to an authenticated legal text. This project builds directly on the RAG framework, extending it with domain-specific adaptations including reranking, hallucination prevention, and graph-based relational reasoning.

2.2 VECTOR DATABASES AND SEMANTIC SEARCH

Traditional information retrieval systems rely on keyword matching (BM25, TF-IDF), which fails to capture semantic similarity between a query and a document. A query like "punishment for assault" would not match a legal section that uses the term "causing hurt" unless the system understands their semantic equivalence. Dense vector representations, generated by transformer-based embedding models, address this by mapping both queries and documents to a shared high-dimensional semantic space.

Sinha (2025) describes vector databases as the backbone of modern RAG systems, enabling sub-millisecond approximate nearest-neighbor search across millions of embedded documents. ChromaDB, used in this project, is a lightweight, persistent vector database

optimized for local deployment, making it ideal for a legal AI system that must operate on sensitive, confidential documents without external API calls for retrieval.

2.3 GRAPH-BASED RAG (GRAPH-RAG)

Peng et al. (2024) conducted a comprehensive survey on Graph Retrieval-Augmented Generation, establishing that vector search alone is insufficient for tasks requiring multi-hop reasoning—where the answer to a question depends on traversing a chain of related documents or entities. In the legal domain, this is particularly relevant: understanding whether an offense is cognizable requires knowing not just the offense definition, but its procedural classification, which may reference another section entirely.

Han et al. (2024) specifically demonstrated that augmenting RAG with graph structures enables the AI to reason about hierarchical and cross-referential relationships between documents. Their work showed significant improvements in multi-hop question answering benchmarks. This project implements GraphRAG using the NetworkX library, constructing a directed multigraph of Indian legal sections with over 1,140 nodes and 1,860 edges encoding PART_OF_CHAPTER and REFERENCES relationships.

2.4 HALLUCINATION IN LEGAL AI

Hallucination—the generation of factually incorrect but syntactically plausible content—is a well-documented problem in LLMs. In the legal domain, hallucination is not merely an accuracy issue but a safety and liability concern. An AI system that generates a fabricated section number could cause a legal professional to act on non-existent law. Research in this area has focused on grounding strategies (RAG), output validation, and uncertainty quantification. This project implements a Regex-based citation verification system that cross-checks every legal reference in the LLM output against the ChromaDB metadata, flagging any hallucinated section numbers.

2.5 HYBRID RETRIEVAL SYSTEMS

The hybrid RAG systems research documented on ResearchGate (2025) demonstrates that combining semantic vector search with structural approaches yields superior results compared to either method in isolation. Vector search excels at finding semantically similar content but struggles with exact matching and structural reasoning. Graph-based search excels at structural and relational queries but requires a well-defined schema. Hybrid systems that fuse both modalities have shown state-of-the-art performance on complex retrieval benchmarks, motivating the hybrid architecture adopted in this project.

2.6 GAPS IN EXISTING LITERATURE

While significant work exists on general RAG systems and legal AI in the US and EU contexts, there is a notable gap in research addressing Indian criminal law specifically, particularly the newly enacted BNS, BNSS, and BSA frameworks introduced in 2023. Existing Indian legal AI systems predominantly focus on the older IPC/CrPC structure and do not incorporate the structural evolution of the new laws. Furthermore, most existing legal AI systems lack robust hallucination prevention mechanisms, relying solely on RAG grounding without output validation. This project addresses both gaps by building a system specifically tailored to BNS/BNSS/BSA with an explicit hallucination verification layer.

Chapter 3: System Architecture

3.1 ARCHITECTURE OVERVIEW

The AI Lawyer Pro system follows a microservices-based architecture, separating the user-facing frontend from the AI processing backend. This design choice ensures scalability, maintainability, and the ability to independently update or replace individual components without affecting the entire system. The architecture is composed of four primary layers: the Data Layer, the Retrieval Layer, the Reasoning Layer, and the Presentation Layer.

Figure 3.1 illustrates the high-level system architecture:

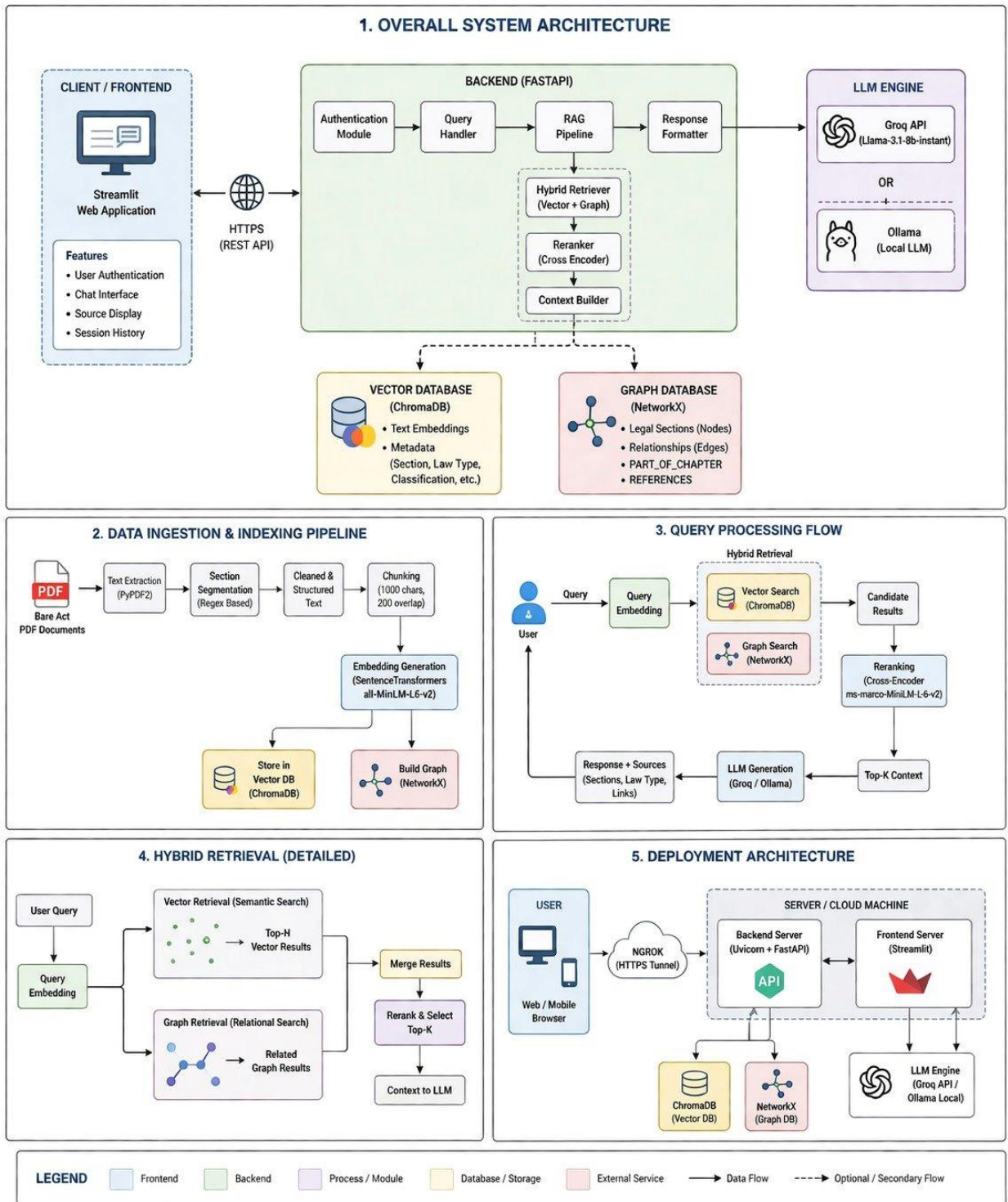
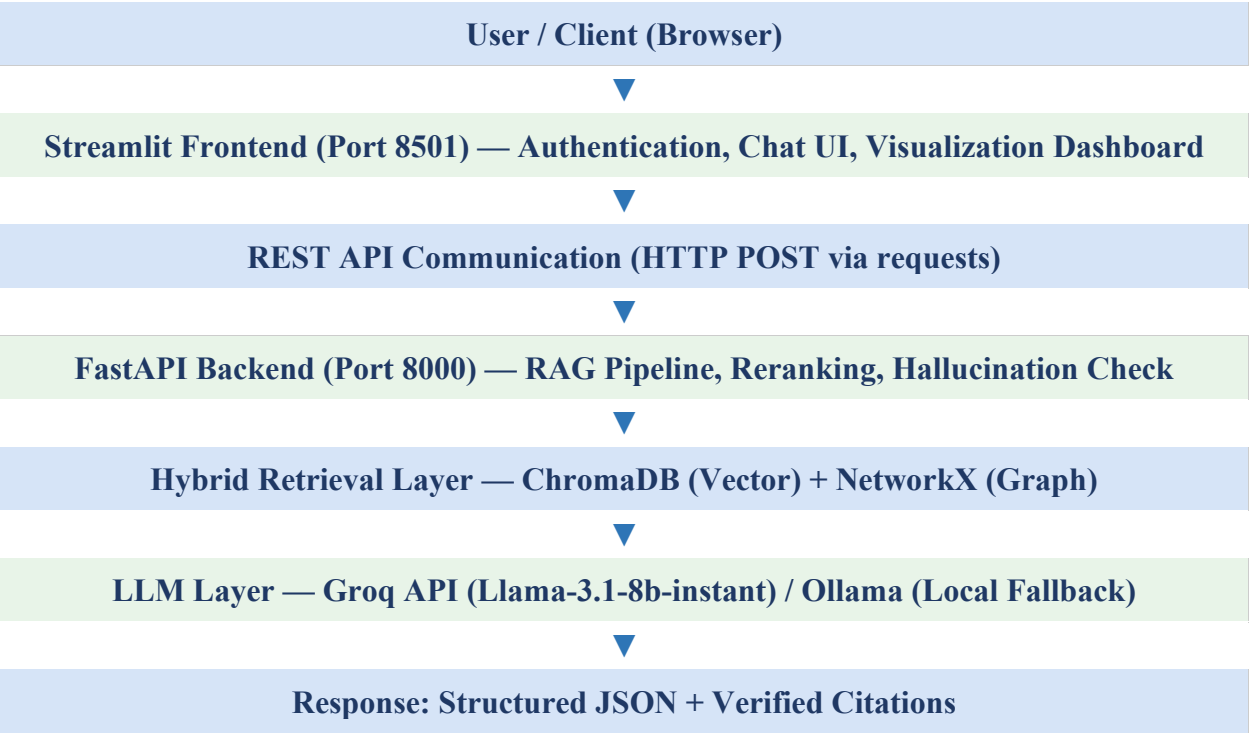


Figure 3.1: System Architecture – Microservices Overview



3.2 TECHNOLOGY STACK

Table 3.1 provides a comprehensive overview of all technologies used in the system:

Technology	Layer	Purpose
PyPDF2	Data Engineering	Raw text extraction from Official Bare Act PDFs
Regex (Python re)	Data Engineering	Intelligent section boundary detection and segmentation
SentenceTransformers	Embeddings	all-MiniLM-L6-v2 for dense vector generation
ChromaDB	Vector Database	Persistent local semantic search database
Cross-Encoder	Reranking	ms-marco-MiniLM-L-6-v2 for precision reranking
NetworkX	GraphRAG	Directed multigraph for legal relationship modeling
Groq API	LLM	Llama-3.1-8b-instant for fast, high-quality inference
Ollama	LLM Fallback	Local offline model execution

Technology	Layer	Purpose
FastAPI	Backend	High-performance REST API for AI pipeline
Uvicorn	Backend Server	ASGI server running on port 8000
Streamlit	Frontend	Interactive web application and chat interface
Plotly	Visualization	Dynamic charts: scatter, gauge, bar
pdfplumber	OCR Input	Enhanced PDF text extraction
Tesseract	OCR Engine	Optical Character Recognition for scanned PDFs
poppler	OCR Support	PDF rendering for Tesseract image conversion
Ngrok	Deployment	Secure tunneling for public demo access
venv	Environment	Python virtual environment for dependency isolation

3.3 DATA FLOW

The complete data flow from user input to system response follows this sequence:

Figure 3.2: Data Engineering Pipeline - PDF to Vector DB

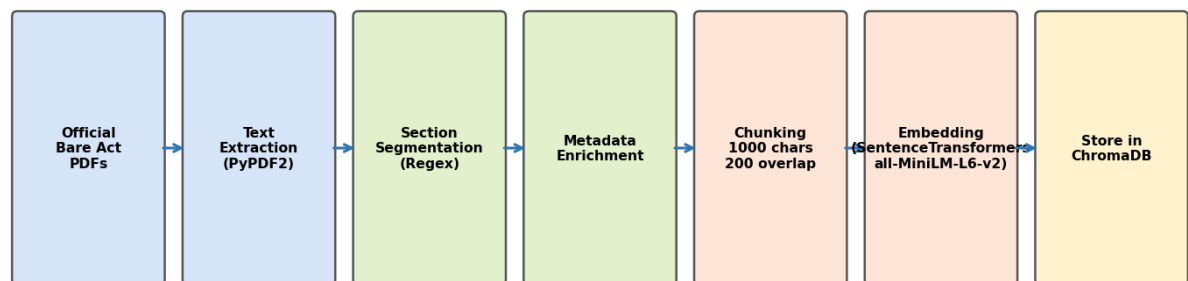


Figure 3.2: Data Engineering Pipeline – PDF to Vector DB

9. User uploads a legal document (FIR) through the Streamlit interface
10. Document is processed through the dual-pipeline reader: standard text extraction first, OCR fallback if needed
11. User query is converted to a dense vector embedding using SentenceTransformers
12. ChromaDB performs semantic similarity search, retrieving top-K relevant legal chunks

13. Cross-Encoder reranker re-evaluates and reorders the retrieved chunks for maximum precision
14. GraphRAG performs a parallel graph traversal to find relationally connected legal sections
15. Retrieved context (vector + graph results) is assembled into a structured prompt
16. Groq API (LLM) generates a structured JSON response with citations, timeline, risk score, and section impact
17. Citation verification system cross-checks all referenced sections against ChromaDB metadata
18. Verified response is returned to Streamlit frontend and rendered as visualizations

Chapter 4: Data Engineering and Preprocessing

4.1 OVERVIEW

The foundation of any reliable AI system is the quality and integrity of its underlying data. In the legal domain, this is especially critical: a single incorrectly parsed section boundary, a missing metadata field, or a corrupted text extraction can lead to the system retrieving the wrong law entirely. The data engineering phase of AI Lawyer Pro was therefore designed with extreme care, prioritizing accuracy and completeness over convenience.

4.2 DATA SOURCE SELECTION – A CRITICAL DECISION

The project initially evaluated CSV-based datasets available online for IPC and CrPC sections. These datasets appeared convenient—pre-structured, easy to load, and ready for embedding. However, a careful audit revealed critical deficiencies:

- Missing procedural metadata: Bailable vs. Non-Bailable classification was absent or incomplete
- No Cognizable vs. Non-Cognizable flags, which are fundamental to arrest procedure
- Inconsistent section numbering and truncated content in several sections
- No coverage of the newly enacted BNS, BNSS, and BSA frameworks

The decision was made to abandon the CSV approach entirely and source all legal content directly from Official Bare Act PDFs—the authoritative, government-published legal texts. This was a pivotal architectural decision that prioritized accuracy over convenience, establishing the principle that the system's knowledge base must be directly traceable to authenticated legal sources.

4.3 TEXT EXTRACTION

The PyPDF2 library was used for initial text extraction from the Bare Act PDFs. The extraction process produced raw, unstructured text containing the complete content of the legal acts. The raw text required significant cleaning and structuring before it could be used

for retrieval, as PDFs often contain formatting artifacts, header/footer repetitions, and page numbering sequences embedded within the legal text.

4.4 SECTION SEGMENTATION

Legal documents follow a specific structural pattern where each section begins with a section number (e.g., "Section 103." or "103.") followed by the section title and content. However, this structure is not perfectly clean in raw PDF extractions due to line breaks, hyphenation, and formatting inconsistencies.

A custom Regex-based parsing engine was developed to identify section boundaries precisely. The Regex patterns were designed to:

- Detect the start of a new section based on numbering patterns
- Handle both numbered sections and sub-sections (e.g., 103(1), 103(2))
- Capture section titles where present
- Handle edge cases such as sections spanning multiple pages

4.5 METADATA ENRICHMENT

Beyond extracting the raw text of each section, the data pipeline enriched each section with critical procedural metadata:

- law_type: BNS, BNSS, or BSA
- section: The section number
- classification: Bailable/Non-Bailable, Cognizable/Non-Cognizable where applicable
- punishment_duration: Imprisonment term where specified

This metadata enrichment was essential for the citation verification system and for enabling metadata-based filtering in ChromaDB retrieval queries.

4.6 CHUNKING STRATEGY

After section segmentation, each legal section was further chunked for embedding into the vector database. The chunking parameters were carefully chosen:

Parameter	Value / Explanation
Chunk Size	1,000 characters
Overlap	200 characters
Rationale	Prevent breaking legal definitions across chunks
Context Preservation	200-char overlap ensures continuity between adjacent chunks
Challenge Addressed	Legal definitions often span multiple sentences; naive splitting loses context

The 200-character overlap is particularly important for legal text, where a definition in one sentence may reference a term defined in the previous sentence. Without overlap, a chunk boundary between these sentences would result in an incomplete, ambiguous chunk that could mislead the retrieval system.

Figure 4.1: Chunking Strategy - 1000 char / 200 overlap

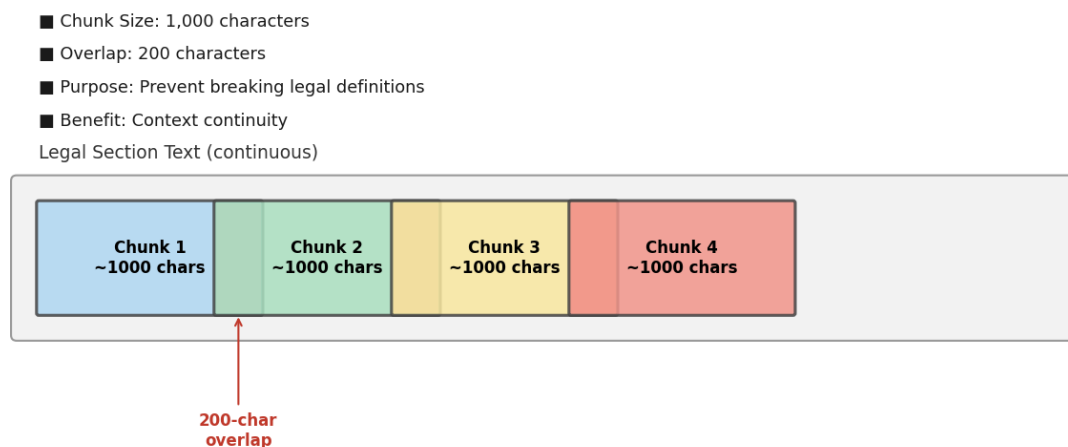


Figure 4.1: Chunking Strategy – 1000 char / 200 overlap

Chapter 5: Semantic Representation and Vector Database

5.1 EMBEDDING MODEL

Converting legal text into dense vector representations is the cornerstone of the semantic retrieval system. The SentenceTransformers library was used with the all-MiniLM-L6-v2 model. This model was chosen for the following reasons:

- Efficiency: The model produces 384-dimensional embeddings with very fast inference, suitable for real-time query processing
- Quality: Despite its compact size, all-MiniLM-L6-v2 produces high-quality semantic representations that capture meaning rather than just keywords
- Local execution: The model runs entirely on-device, ensuring that sensitive legal documents are never sent to external embedding APIs

Figure 5.1: Vector Embedding Flow - SentenceTransformers to ChromaDB

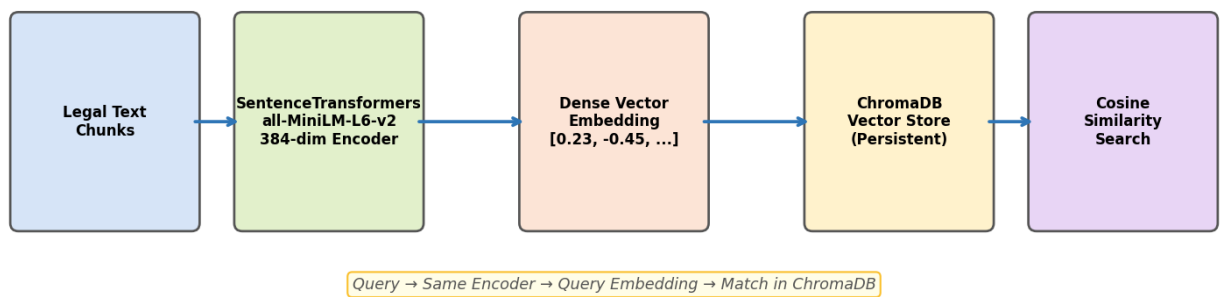


Figure 5.1: Vector Embedding Flow – SentenceTransformers to ChromaDB

Both the legal document chunks (at indexing time) and user queries (at inference time) are converted to embeddings using this same model, enabling cosine similarity comparison in the shared vector space.

5.2 CHROMADB SETUP

ChromaDB was selected as the vector database for the following reasons:

- Persistent local storage: ChromaDB stores embeddings on disk, surviving application restarts without reindexing
- Metadata filtering: ChromaDB supports filtering retrieval results by metadata fields, enabling law_type and classification-based filtering
- Lightweight deployment: No external database server required; ChromaDB runs as a Python library
- Active development: Well-maintained with a clean Python API

5.3 DATABASE SCHEMA

Each entry in ChromaDB consists of three components:

Component	Description
Document (Text)	The legal text chunk (1,000 characters with 200-char overlap)
Embedding (Vector)	384-dimensional float vector from SentenceTransformers
Metadata	law_type, section, classification, punishment_duration

5.4 WHY SEMANTIC SEARCH OVER KEYWORD SEARCH

Traditional keyword search systems like BM25 or TF-IDF match documents based on the presence of exact query terms. In the legal domain, this approach fails because:

- Legal terminology is highly specialized and non-intuitive: A layperson querying "robbery" may not know that the relevant BNS section uses the term "dacoity" for gang robbery
- Synonymous concepts are expressed differently: "Murder" and "culpable homicide" have distinct legal meanings but semantically overlap in common usage
- Multi-lingual input: Users may query in informal Hindi-English mixed language

Semantic search resolves these issues by matching based on meaning rather than exact terms, dramatically improving retrieval relevance for legal queries. This fundamental shift from keyword to semantic search was a core design decision that enables AI Lawyer Pro to serve users who do not know precise legal terminology.

Chapter 6: Basic RAG Pipeline

6.1 PIPELINE OVERVIEW

With the vector database populated, the first version of the RAG pipeline was built. This baseline system established the fundamental query-retrieve-generate flow that subsequent phases would build upon and refine.

The basic RAG pipeline operates as follows:

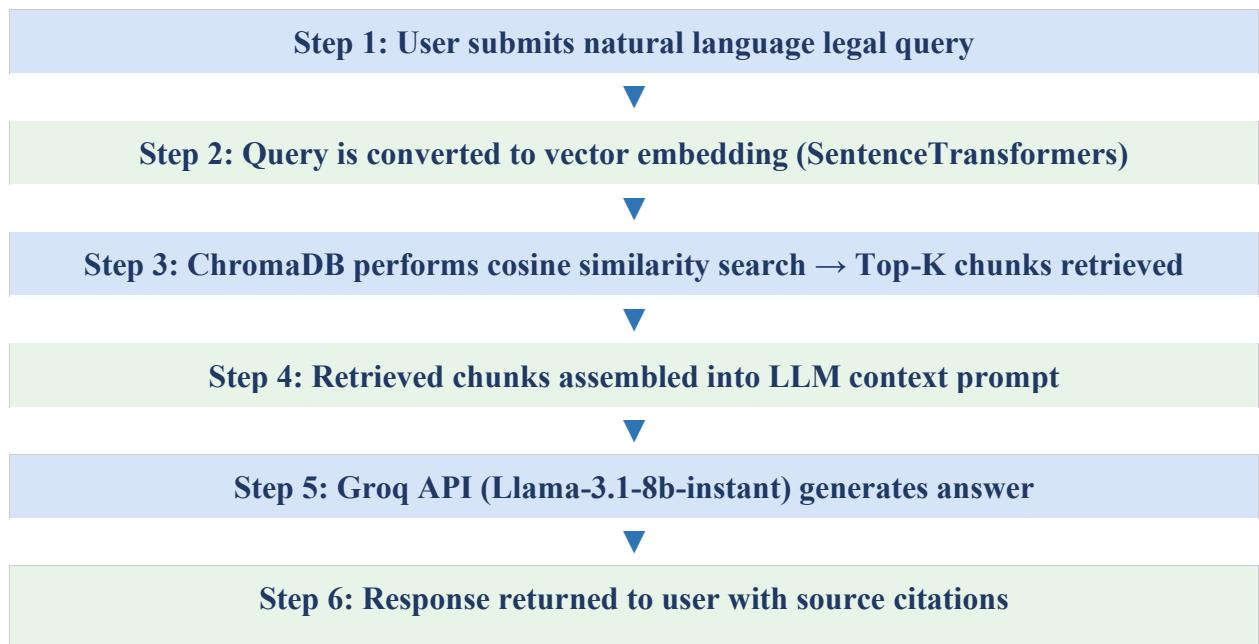


Figure 6.1: Basic RAG Pipeline Flow

6.2 RETRIEVAL CONFIGURATION

The initial retrieval configuration retrieved the top 5 most similar chunks from ChromaDB for each query. The cosine similarity threshold was set conservatively to avoid retrieving tangentially related but ultimately irrelevant legal sections. Retrieved chunks included their full metadata, enabling the system to provide source attribution (law type, section number) alongside every generated answer.

6.3 LIMITATIONS IDENTIFIED

Testing the basic RAG pipeline revealed a critical limitation: while the retrieved chunks were generally relevant to the query, they were not always optimally ordered. The cosine

similarity metric used by ChromaDB measures the geometric similarity between query and document embeddings, but this does not perfectly capture "which chunk most directly answers this specific query." Consequently:

- Some highly relevant chunks appeared at position 4 or 5, while less useful chunks occupied the top positions
- The LLM, given a less-than-optimal context ordering, occasionally focused on the first chunks (which may be less relevant) and generated suboptimal answers
- Precision needed improvement: the system could be "approximately right" but not "precisely correct"

This limitation motivated the development of the Reranking Mechanism described in Chapter 7.

Chapter 7: Reranking Mechanism

7.1 THE PROBLEM WITH BI-ENCODER RETRIEVAL

The embedding model used in the basic RAG pipeline (all-MiniLM-L6-v2) is a bi-encoder: it encodes the query and each document independently into vectors, and retrieval is performed by computing the distance between these independent encodings. This approach is extremely fast—chromaDB can search millions of embeddings in milliseconds—but it is inherently limited because the query and document are encoded in isolation, without any cross-attention between them.

A cross-encoder, by contrast, takes the query and a candidate document as a joint input and computes a relevance score through full cross-attention. This is computationally more expensive (cannot be pre-indexed), but produces much more accurate relevance scores because the model can attend to the specific interaction between query terms and document terms.

7.2 CROSS-ENCODER RERANKING IMPLEMENTATION

The solution implemented was a two-stage retrieval process, commonly known as "retrieve and rerank" or "Coarse-to-Fine" retrieval:

Stage 1 (COARSE): ChromaDB retrieves top-20 candidates using fast bi-encoder similarity



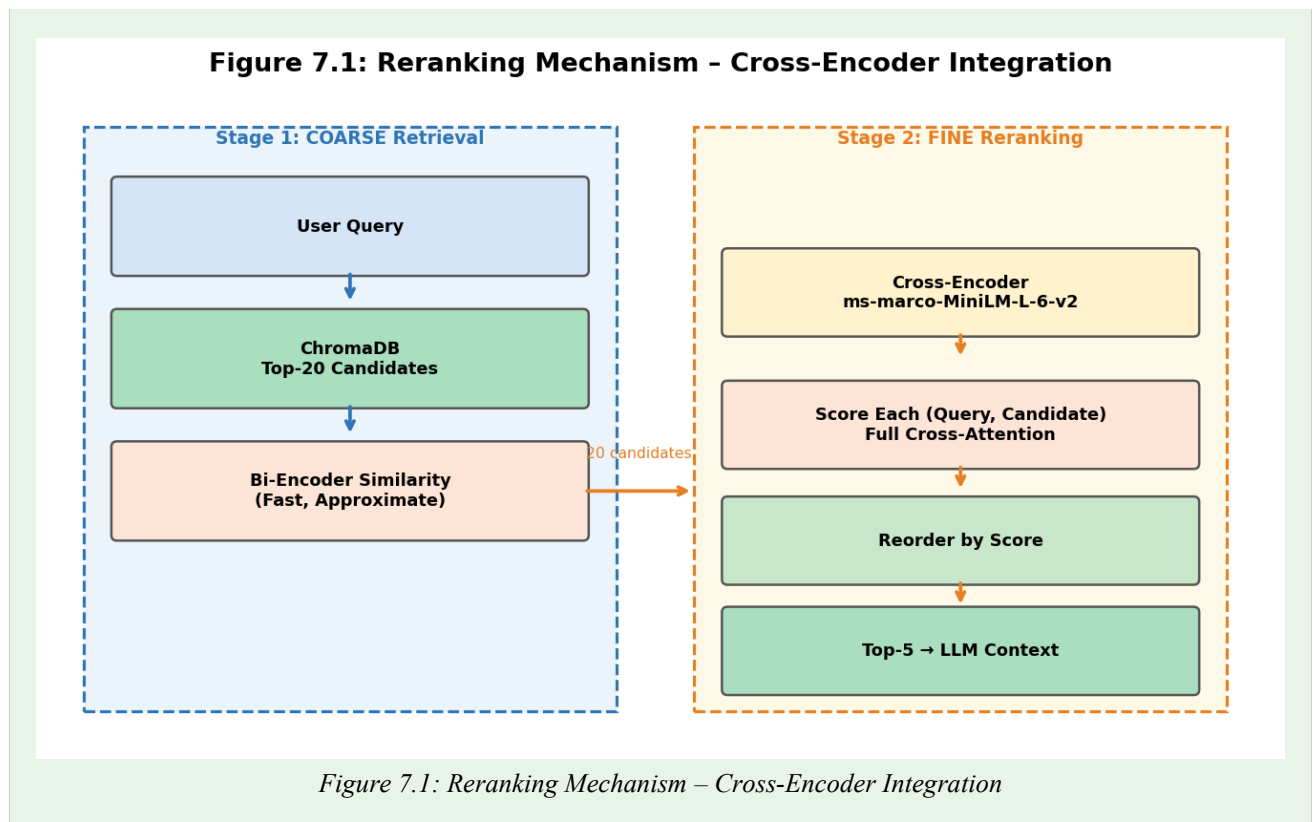
Stage 2 (FINE): Cross-Encoder scores each of the 20 candidates against the query



Stage 3: Candidates reordered by Cross-Encoder score → top-5 selected



Stage 4: Reranked top-5 passed to LLM for answer generation



The Cross-Encoder model used was ms-marco-MiniLM-L-6-v2, fine-tuned on the MS MARCO passage ranking dataset. This model was selected because it was specifically trained to rank passages by relevance to a question—precisely the task required here.

7.3 IMPACT OF RERANKING

The introduction of the reranking layer had a significant positive impact on answer quality:

- Irrelevant chunks that had achieved high cosine similarity scores (due to sharing general legal vocabulary) were correctly demoted by the cross-encoder
- The most precisely relevant chunks—those that directly addressed the specific query—were elevated to the top positions
- LLM answers became more precise and legally accurate, with fewer tangential references

The reranking layer represents the transition from an "approximately right" retrieval system to a "precisely correct" one. This precision is essential in the legal domain where the

difference between Section 103 and Section 104 of BNS can mean the difference between cognizable and non-cognizable offense classification.

Chapter 8: Hallucination Prevention System

8.1 THE HALLUCINATION PROBLEM IN LEGAL AI

Hallucination—where an LLM generates information that sounds plausible but is factually incorrect—is a fundamental limitation of all current LLMs. In most consumer applications, minor hallucinations are an inconvenience. In the legal domain, they are a serious problem. An LLM that generates a fabricated BNS section number in response to a legal query could cause a lawyer to argue a non-existent provision, a defendant to be misadvised about their rights, or a judge to be presented with incorrect legal references.

While the RAG approach significantly reduces hallucination by grounding the LLM in retrieved context, it does not eliminate it. LLMs can still:

- Invent section numbers that do not exist in the retrieved context
- Modify section numbers slightly (e.g., citing Section 103(2) when only Section 103(1) exists)
- Blend information from different sections into a fabricated hybrid section

8.2 CITATION VERIFICATION SYSTEM

To address this, AI Lawyer Pro implements a post-generation citation verification system. After the LLM generates a response, every legal section number cited in the response is extracted using Regex patterns and cross-checked against the ChromaDB metadata.

The verification process operates as follows:

19. Regex extraction: All section references (e.g., "Section 103", "BNS Section 45", "103(2)") are extracted from the LLM output using a comprehensive pattern set
20. Database lookup: Each extracted reference is queried against ChromaDB metadata to verify existence
21. Comparison: The cited section content is compared against what the LLM claims about that section
22. Flagging: Any citation not found in the database, or where the LLM's description contradicts the actual section content, is flagged as a potential hallucination

23. Response annotation: The final response is annotated to indicate which citations are verified and which are flagged

Figure 8.1: Hallucination Prevention System - Citation Verification

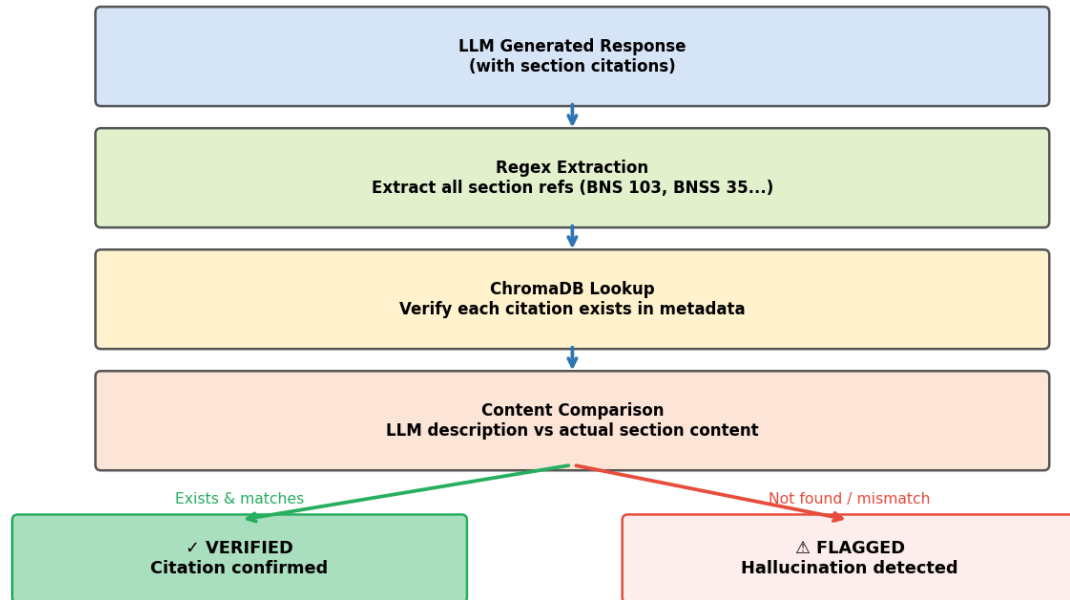


Figure 8.1: Hallucination Prevention System – Citation Verification

8.3 ENGINEERING SIGNIFICANCE

The citation verification system represents a crucial safety layer that transforms AI Lawyer Pro from a "best-effort" AI assistant into a system with active output validation. This architectural decision reflects a fundamental principle: in high-stakes domains, the AI system must not only attempt to be correct but must also actively verify its own outputs. The implementation demonstrates mature AI engineering thinking that goes beyond basic LLM integration to build a trustworthy, production-grade system.

Chapter 9: GraphRAG – Graph-Based Retrieval

9.1 MOTIVATION FOR GRAPH-BASED RETRIEVAL

Vector search, as implemented in ChromaDB, is powerful for finding semantically similar content. However, it has a fundamental limitation: it treats each document chunk as an independent unit and cannot model the structured relationships between legal sections. Consider the following scenario: A user asks about the procedure for arresting someone for an offense under Section 103 of BNS (Murder). The correct answer requires knowing not just the offense definition, but also:

- Which chapter Section 103 belongs to (PART_OF_CHAPTER relationship)
- Which BNSS sections govern arrest procedure for cognizable offenses (REFERENCES relationship)
- Whether bail is applicable, which references another section of BNSS (multi-hop reasoning)

A pure vector search cannot traverse these relational chains. It can only find the sections most semantically similar to the query words—it cannot follow the logical legal reasoning chain that a lawyer would follow. GraphRAG addresses this limitation.

9.2 LEGAL KNOWLEDGE GRAPH CONSTRUCTION

A Directed Multigraph was constructed using the NetworkX library to represent the structural relationships between Indian legal sections. The graph construction process involved:

9.2.1 NODE CREATION

Each legal section from BNS, BNSS, and BSA was represented as a node in the graph. Each node stores:

- Section number and law type
- Section title
- Complete section content

- Procedural metadata (bailable, cognizable, punishment)

9.2.2 EDGE CREATION

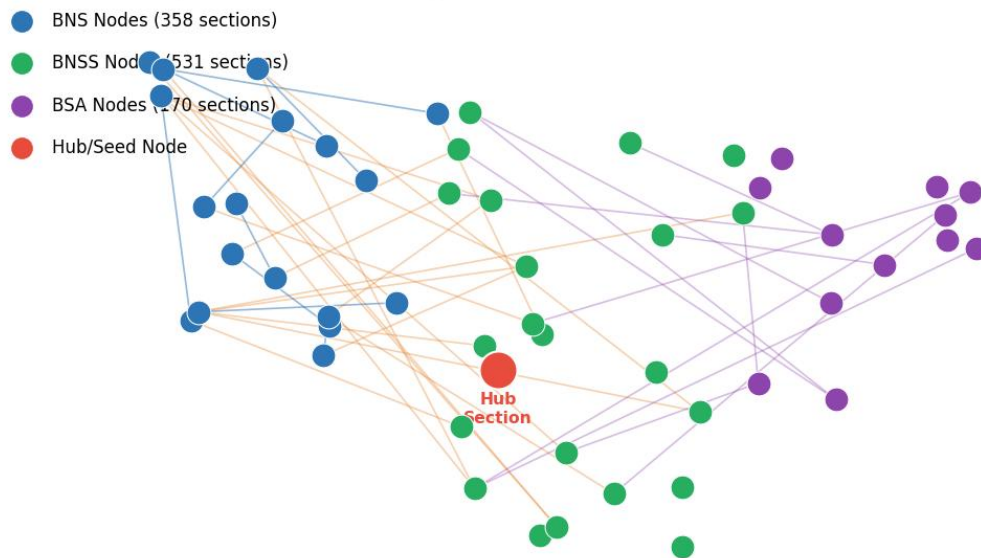
Edges represent the structural and cross-referential relationships between sections:

Edge Type	Meaning
PART_OF_CHAPTER	Links sections to their parent chapter, enabling chapter-level queries
REFERENCES	Links sections that explicitly reference other sections in the text
FOLLOWED_BY	Sequential ordering of sections within a chapter
PROCEDURE_UNDER	Links offense sections (BNS) to their procedural sections (BNSS)

9.2.3 GRAPH STATISTICS

Metric	Value
Total Nodes	1,140+ (legal sections across BNS, BNSS, BSA)
Total Edges	1,860+ (structural and referential relationships)
Graph Type	Directed Multigraph (multiple edge types between same nodes)
Primary Relationship Types	PART OF CHAPTER, REFERENCES
Library Used	NetworkX (Python)
Storage Format	legal_graph.json (serialized graph)

Figure 9.1: GraphRAG Node-Edge Structure (1140 Nodes, 1860 Edges)



Total: 1,140+ Nodes | 1,860+ Edges | Directed Multigraph (NetworkX)

Figure 9.1: GraphRAG Node-Edge Structure (1140 Nodes, 1860 Edges)

9.3 GRAPH TRAVERSAL FOR QUERY ANSWERING

When a query is received, the GraphRAG component performs the following steps:

24. Identify seed nodes: The top sections retrieved by vector search are used as seed nodes in the graph
25. Neighborhood expansion: The graph is traversed to find sections related to the seed nodes via PART_OF_CHAPTER and REFERENCES edges
26. Multi-hop reasoning: The traversal follows multi-hop chains (e.g., offense section → arrest procedure section → bail classification section)
27. Context enrichment: The content of all visited nodes is added to the LLM context, enriching the response with relational legal knowledge

This graph traversal capability transforms AI Lawyer Pro from a system that can find relevant sections into a system that can reason about the legal structure and dependencies

between sections—a capability that is essential for answering complex, multi-aspect legal queries.

Figure 9.2: Legal Graph Relationships - PART_OF_CHAPTER & REFERENCES

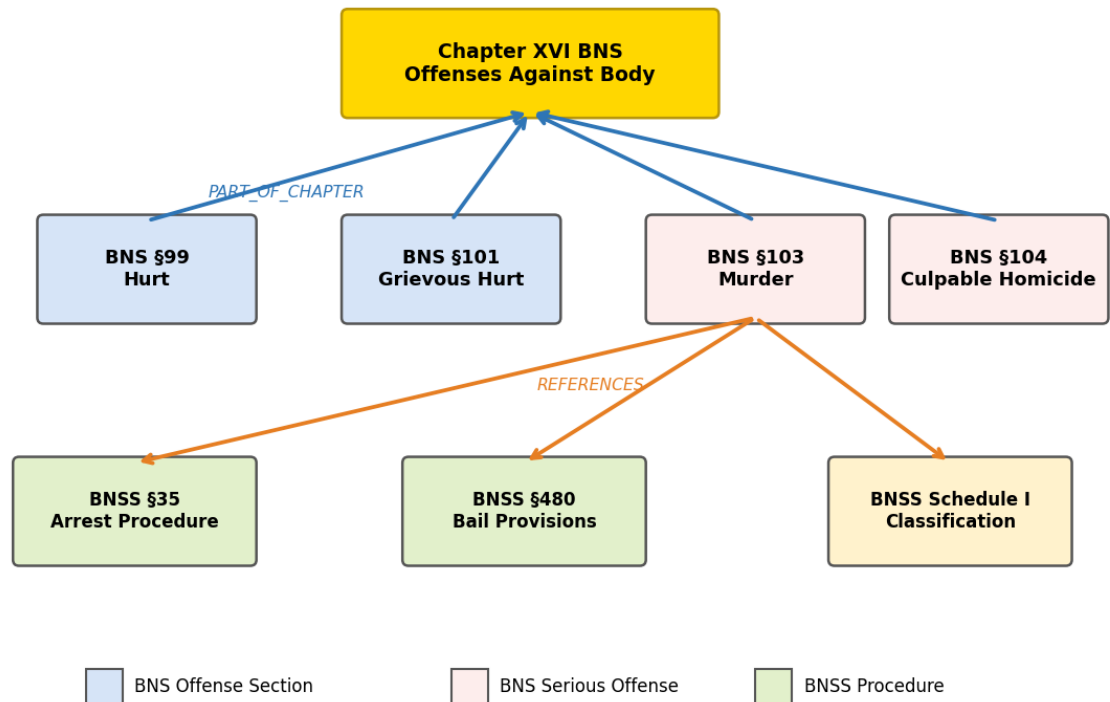
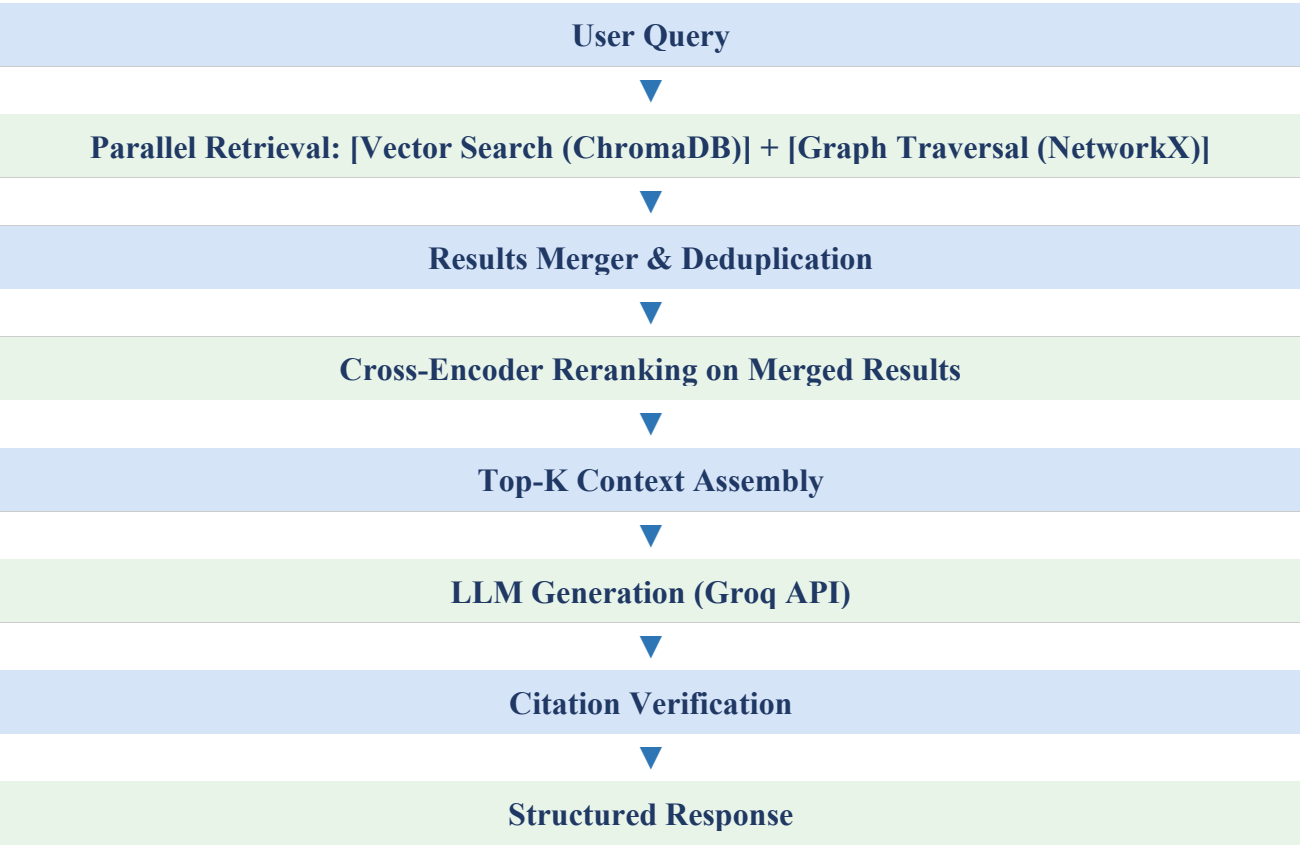


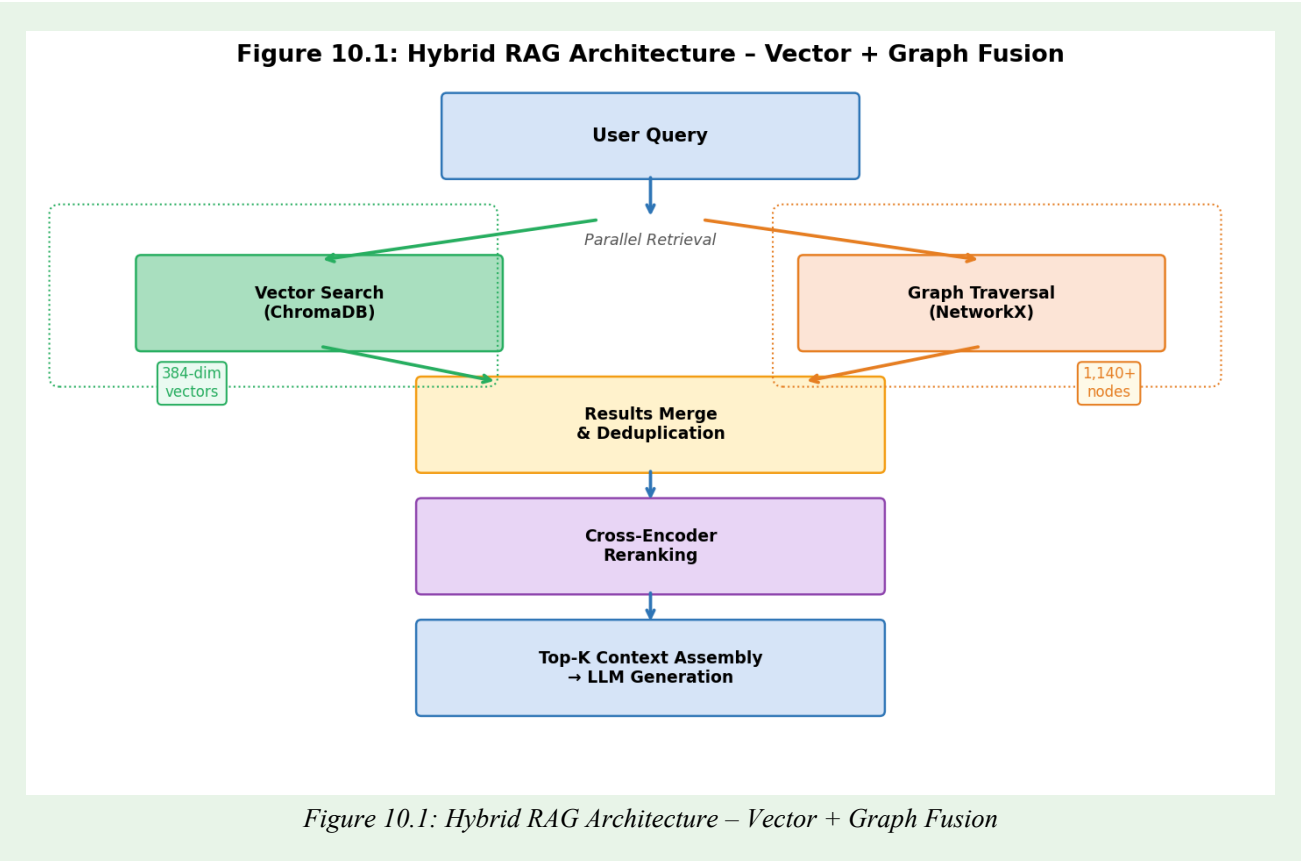
Figure 9.2: Legal Graph Relationships – PART_OF_CHAPTER & REFERENCES

Chapter 10: Hybrid RAG Architecture

10.1 COMBINING VECTOR AND GRAPH RETRIEVAL

The Hybrid RAG architecture represents the culmination of the retrieval engineering work in this project. It combines the semantic power of ChromaDB vector search with the relational intelligence of NetworkX GraphRAG into a unified retrieval pipeline. Both retrieval modalities are executed in parallel for each query, and their results are merged and deduplicated before being passed to the LLM.





10.2 COMPARISON: TRADITIONAL RAG VS HYBRID RAG

Dimension	Traditional RAG	Hybrid RAG
Retrieval Method	Semantic vector similarity	Vector + Graph traversal
Relationship Awareness	None (treats chunks independently)	Full (models PART_OF_CHAPTER, REFERENCES)
Multi-Hop Reasoning	Not supported	Supported via graph traversal
Context Richness	Semantically similar chunks only	Semantically similar + relationally connected chunks
Query Types Handled	Semantic similarity queries	Semantic + structural + relational queries
Hallucination Risk	Higher (less context)	Lower (richer, cross-validated context)

10.3 ENGINEERING RATIONALE

The decision to implement a Hybrid RAG architecture rather than optimizing only the vector search component reflects a deep understanding of the legal domain's requirements. Legal reasoning is inherently relational—laws reference other laws, procedures reference offenses, and punishments reference classifications. A system that can only do semantic search is fundamentally limited in its ability to answer complex legal queries. The hybrid approach enables AI Lawyer Pro to serve as a genuine legal reasoning assistant, not just a legal document search engine.

Chapter 11: Advanced Prompt Engineering

11.1 THE IMPORTANCE OF PROMPT ENGINEERING IN LEGAL AI

Prompt engineering—the design of the instructions and context provided to an LLM—is as important as the retrieval architecture in determining the quality of the final output. A poorly designed prompt can cause an LLM to generate unstructured, verbose, or inconsistent responses even when the retrieved context is perfectly relevant. In a legal AI system that must produce structured JSON for downstream processing (visualization, citation extraction), prompt engineering is critical.

11.2 ZERO-TEMPERATURE INFERENCE

The LLM temperature parameter controls the randomness of the output. At temperature = 1.0, the model samples from the full probability distribution over possible next tokens, producing varied, creative outputs. At temperature = 0.0, the model always selects the most probable next token, producing deterministic, focused outputs.

For AI Lawyer Pro, temperature was set to 0.0 for all legal analysis queries. The rationale:

- Legal analysis must be deterministic: the same FIR analyzed twice must yield the same sections, timeline, and risk score
- Creativity is dangerous in legal AI: a creative LLM may invent plausible-sounding but incorrect legal interpretations
- Structured output requires consistency: the JSON schema enforcement depends on the model reliably generating the same structure

11.3 RELEVANCY FILTERING

The system prompt instructs the LLM to actively filter out irrelevant retrieved context. Despite the reranking layer ensuring high-quality context, some retrieved chunks may be tangentially related but not directly applicable to the specific FIR being analyzed. The system prompt includes an explicit instruction: if a retrieved legal section is not directly relevant to the facts in the uploaded document, it must be ignored and not cited in the

response. This "context contamination" prevention is particularly important in legal analysis where citing an irrelevant section could mislead the legal professional.

11.4 STRUCTURED OUTPUT SCHEMA ENFORCEMENT

The most technically sophisticated prompt engineering technique used in this project is the enforcement of a strict JSON output schema using few-shot prompting. The system prompt includes a hard-coded JSON blueprint within `[[DATA]]` tags that specifies the exact structure of the expected output:

Required JSON Output Schema:

```
{
  "sections_applicable": [
    {"section": "BNS 103", "title": "...", "relevance": "..."}
  ],
  "timeline": [
    {"date": "DD-MM-YYYY", "event": "...", "significance": "..."}
  ],
  "risk_score": 0.0 to 10.0,
  "section_impact": [
    {"section": "...", "impact_weight": 0.0 to 1.0}
  ],
  "summary": "Concise legal analysis...",
  "verified_citations": ["BNS 103", "BNSS 35"]
}
```

By providing this schema as a few-shot example within the prompt, the LLM learns to produce output that exactly matches this structure. This enables the downstream Plotly

visualization system to reliably parse the response into timeline plots, gauge charts, and bar charts without requiring defensive parsing logic for arbitrary output formats.

Figure 11.1: Prompt Engineering Schema - Few-Shot JSON Output

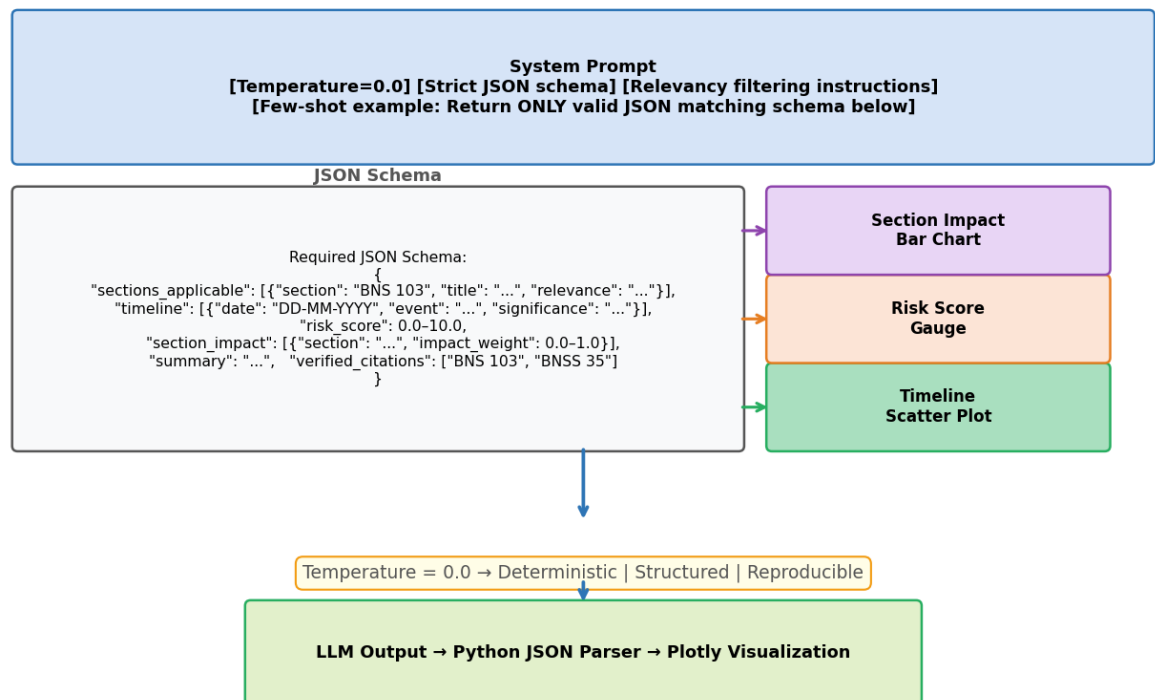


Figure 11.1: Prompt Engineering Schema – Few-Shot JSON Output

11.5 ERROR HANDLING FOR SCHEMA DRIFT

Despite the strict schema enforcement, the LLM occasionally produces minor deviations—for example, using "timestamp" instead of "date" as a JSON key, or omitting a field under certain edge cases. The system implements Python-side auto-correction logic that:

- Auto-renames common key drift patterns (e.g., "timestamp" → "date", "score" → "risk_score")
- Validates that all required keys are present, using safe default values for missing fields
- Prevents Pandas and Plotly from throwing `ValueErrors` due to unexpected data structures

Chapter 12: Frontend Development and Visualization

12.1 STREAMLIT FRONTEND

The user interface was developed using Streamlit, a Python library for rapid web application development that enables the creation of interactive AI applications with minimal frontend code. Streamlit was chosen over alternatives like React or Vue.js because it allows the AI backend logic and the UI to be developed in the same Python environment, dramatically reducing development complexity for a research project of this nature.

12.2 AUTHENTICATION SYSTEM

A secure login/signup system was implemented using Streamlit's session state mechanism. User credentials are stored in a JSON file with appropriate security measures. The authentication system:

- Maintains user-specific session states that persist across page refreshes
- Stores individual user chat histories in separate session containers
- Prevents unauthorized access to the legal analysis dashboard
- Provides a clean, intuitive login/signup interface as the entry point

12.3 CHAT INTERFACE

The primary user interaction happens through a ChatGPT-style conversational interface. Users can:

- Upload FIR documents directly through the chat interface
- Ask natural language questions about the uploaded document or about Indian criminal law in general
- View AI responses in a clean, formatted chat bubble layout
- Access a dynamic "Sources" tab beneath each response that shows exactly which BNS/BNSS/BSA sections were used to generate the answer

The Sources tab is a distinctive feature that directly addresses the transparency and trust requirements of legal AI. By showing the exact legal sections retrieved, the system enables legal professionals to independently verify every claim made by the AI.

Figure 12.1: Streamlit UI - Chat Interface with Sources Tab

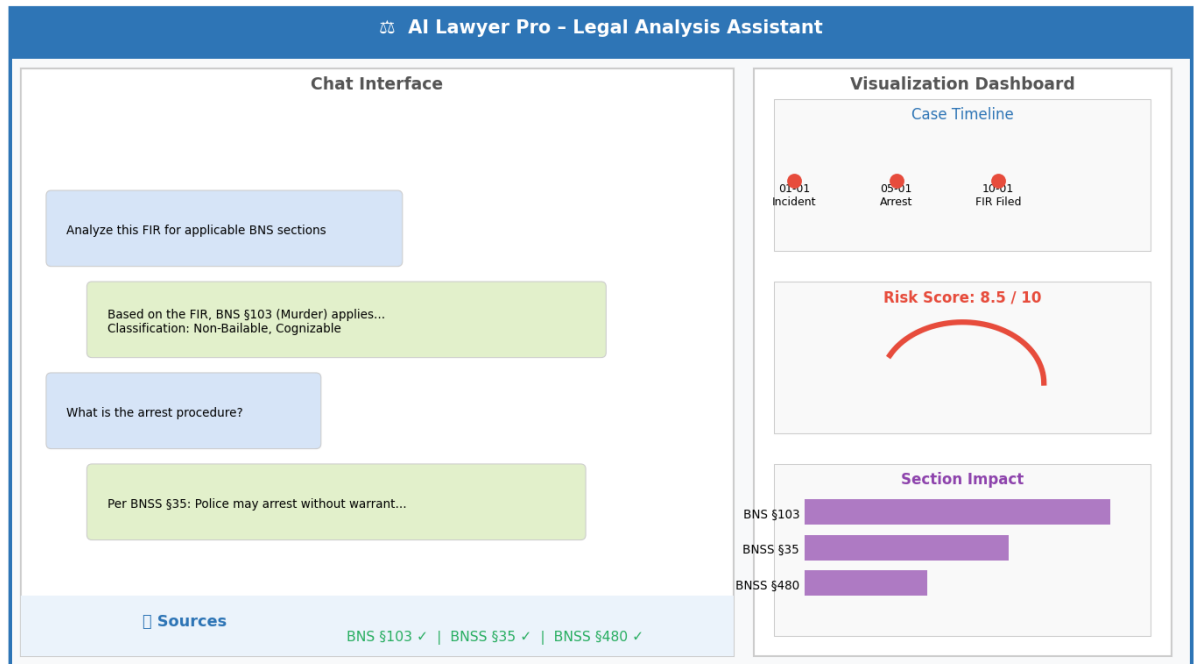


Figure 12.1: Streamlit UI – Chat Interface with Sources Tab

12.4 CUSTOM CSS AND LAYOUT

Standard Streamlit layout has limitations for chat applications, particularly around positioning of the input box and preventing the chat history from being obscured by dashboard elements. Custom CSS using Flexbox was written to:

- Create a sticky, right-aligned chat interface that does not overlap the main data dashboard
- Style the assistant and user message bubbles with distinct colors and fonts for clear visual distinction
- Ensure responsive layout across different screen sizes

12.5 PLOTLY VISUALIZATION DASHBOARD

The right panel of the interface features a dynamic analytics dashboard powered by Plotly, populated from the structured JSON output of the LLM. Three visualizations are generated for each analyzed FIR:

12.5.1 CASE TIMELINE – SCATTER PLOT

A Plotly Express scatter plot that maps the chronological sequence of events described in the FIR. Each data point represents a key event with its date and a brief description. The timeline provides an at-a-glance overview of the case progression from the first incident to the FIR filing, helping legal professionals quickly understand the temporal structure of the case.

Figure 12.2: Plotly Visualization - Case Timeline Scatter Plot

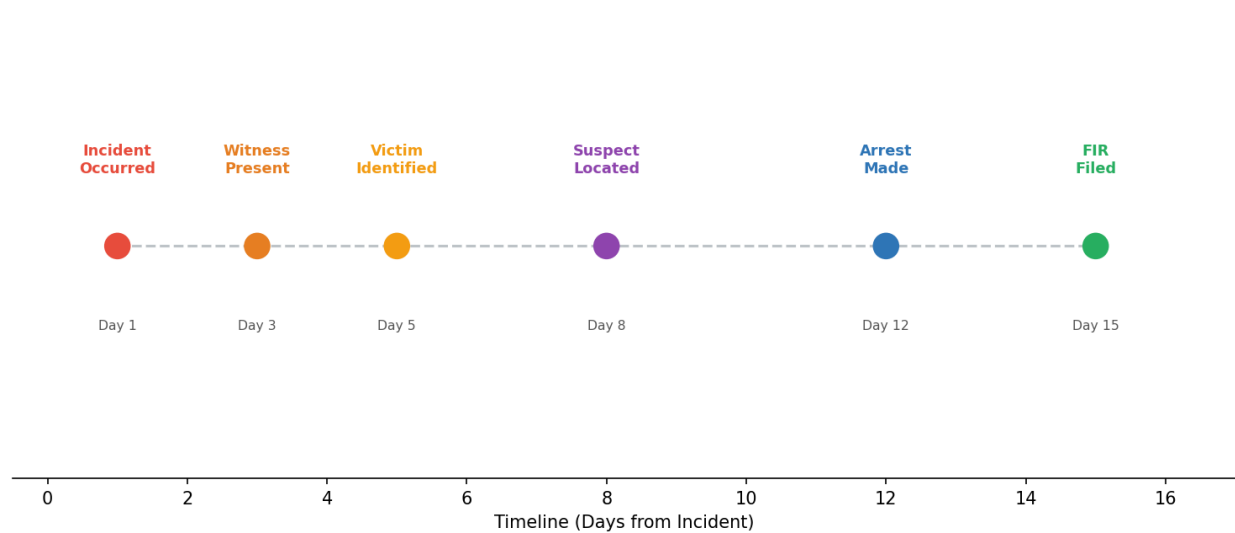


Figure 12.2: Plotly Visualization – Case Timeline Scatter Plot

12.5.2 CASE RISK SCORE – GAUGE CHART

A Plotly gauge chart that displays the overall severity and legal risk score of the case, on a scale of 0 to 10. The score is calculated by the LLM based on the nature of the offenses identified, the number of applicable sections, and the severity of the maximum punishment. The gauge uses color coding: green (0–3), amber (3–7), and red (7–10) to provide immediate visual indication of case severity.

Figure 12.3: Plotly Visualization - Risk Score Gauge Chart



Figure 12.3: Plotly Visualization – Risk Score Gauge Chart

12.5.3 SECTION IMPACT – HORIZONTAL BAR CHART

A Plotly horizontal bar chart that visualizes the relative impact or relevance weight of each identified legal section to the case. Sections with higher relevance scores are shown with longer bars, enabling the legal professional to immediately identify the primary charges versus secondary or supporting provisions.

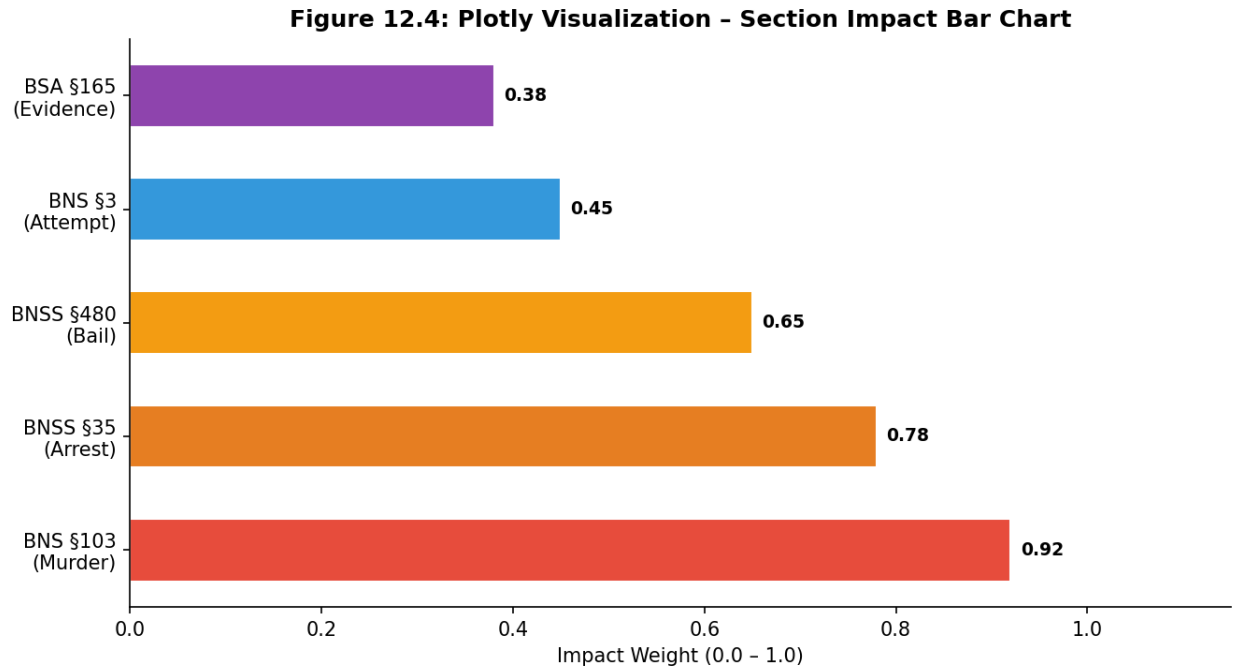


Figure 12.4: Plotly Visualization – Section Impact Bar Chart

Chapter 13: OCR Integration

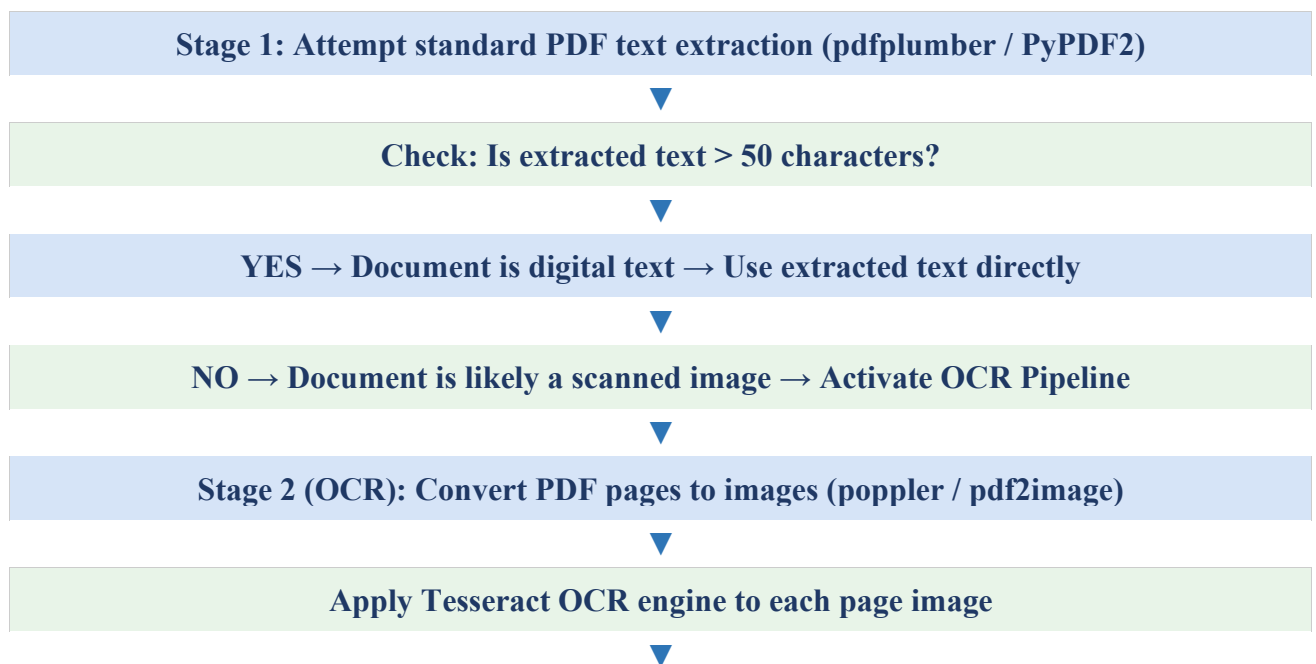
13.1 THE SCANNED PDF PROBLEM

A significant practical challenge in legal document processing is that not all PDFs are created from digital text. Many FIRs and legal documents in Indian police stations and courts are physical documents that have been scanned and saved as PDF files. These scanned PDFs contain images of text rather than actual text characters, making them invisible to standard PDF text extraction tools like PyPDF2 or pdfplumber.

A system that cannot process scanned PDFs would be severely limited in its practical utility—it would be unable to handle a large portion of the actual legal documents that users need to analyze. To address this, a dual-pipeline document reading system was implemented.

13.2 DUAL-PIPELINE ARCHITECTURE

The document reading pipeline operates in two stages:



Combine OCR output into full document text

Proceed with normal processing pipeline

Figure 13.1: OCR Dual-Pipeline - Text Extraction + Tesseract Fallback

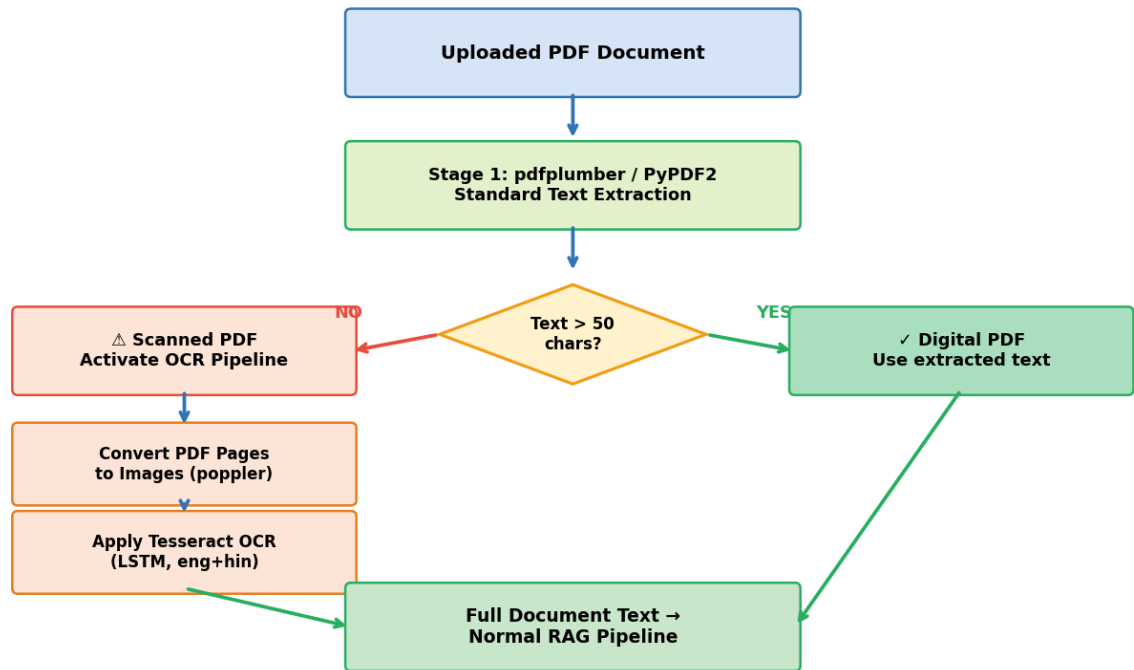


Figure 13.1: OCR Dual-Pipeline – Text Extraction + Tesseract Fallback

13.3 TESSERACT CONFIGURATION

The Tesseract OCR engine was configured with the following settings for optimal performance on legal documents:

- Language: English (eng) as primary, with support for Hindi (hin) for bilingual FIRs
- Page segmentation mode: Automatic page segmentation for mixed-layout documents
- OCR engine mode: LSTM neural network mode for highest accuracy
- Preprocessing: Image binarization and deskewing applied before OCR to handle tilted or poor-quality scans

13.4 IMPACT ON SYSTEM UTILITY

The OCR integration significantly expands the practical utility of AI Lawyer Pro. The system can now process:

- Digitally generated FIRs (PDF with embedded text)
- Scanned physical FIRs (PDF with image content)
- Mixed documents (some pages digital, some scanned)

This dual-pipeline approach ensures that the document reading layer is robust to the full range of document formats encountered in real Indian legal practice.

Chapter 14: Deployment Strategy

14.1 ENVIRONMENT ISOLATION

All project dependencies were isolated in a Python virtual environment (venv) to prevent version conflicts between the numerous libraries used (FastAPI, Torch, ChromaDB, SentenceTransformers, Streamlit, Plotly, etc.). This ensures complete reproducibility: any machine with Python 3.10+ can set up the project by creating the venv and installing requirements from the requirements.txt file.

14.2 MULTI-SERVICE EXECUTION

The microservices architecture requires two processes to run simultaneously:

Component	Command / Detail
Backend (FastAPI)	<code>uvicorn main:app --host 0.0.0.0 --port 8000</code>
Frontend (Streamlit)	<code>streamlit run app.py --server.port 8501</code>
Backend Purpose	Handles all AI processing: RAG, reranking, graph traversal, LLM calls, citation verification
Frontend Purpose	Serves the user interface, handles file uploads, renders visualizations
Communication	Frontend sends POST requests to <code>http://localhost:8000</code> via the requests library

14.3 NGROK TUNNELING FOR LIVE DEMO

For live demonstration and testing, Ngrok was used to create a secure HTTPS tunnel from the local Streamlit server (port 8501) to a public URL. This enabled:

- Sharing the running application with the project guide for live demonstration without deploying to a cloud server
- Remote testing from different devices and networks
- HTTPS encryption for all traffic, ensuring confidentiality of uploaded legal documents during demo sessions

14.4 ENVIRONMENT VARIABLES

Sensitive configuration (API keys for Groq, environment paths) is managed through a `.env` file, loaded at startup using the `python-dotenv` library. This follows security best practices by keeping credentials out of the source code and enabling environment-specific configuration without code changes.

Chapter 15: Legal Framework Transition – IPC/CrPC to BNS/BNSS/BSA

15.1 THE LEGAL REFORM CONTEXT

In 2023, the Government of India introduced three landmark criminal law reforms that replaced the British-era criminal code which had governed Indian law since 1860. These reforms represent the most significant overhaul of Indian criminal law in over 160 years:

New Law	Replaces	Scope
Bharatiya Nyaya Sanhita (BNS)	Replaces Indian Penal Code (IPC) 1860	Substantive criminal law – defines offenses and punishments
Bharatiya Nagarik Suraksha Sanhita (BNSS)	Replaces Code of Criminal Procedure (CrPC) 1973	Procedural criminal law – arrest, trial, bail, investigation
Bharatiya Sakshya Adhiniyam (BSA)	Replaces Indian Evidence Act 1872	Law of evidence – admissibility and proof

15.2 WHY THIS TRANSITION MATTERED

The initial version of AI Lawyer Pro was built on IPC and CrPC datasets. When the new laws were enacted and the old laws were repealed, the system would have become legally irrelevant—it would be analyzing FIRs and providing legal advice based on laws that no longer existed. Transitioning the entire knowledge base and system to BNS/BNSS/BSA was therefore not an optional upgrade but a necessity for the system to remain legally valid.

This transition involved:

- Sourcing and processing the complete Bare Act PDFs for BNS, BNSS, and BSA
- Re-running the entire data engineering pipeline (extraction, segmentation, metadata enrichment, chunking)
- Rebuilding the ChromaDB vector database with the new legal sections
- Reconstructing the NetworkX knowledge graph with updated node and edge relationships
- Updating all system prompts to reference BNS/BNSS/BSA section numbers and terminology
- Testing the complete pipeline with sample FIRs filed under the new legal framework

15.3 SYSTEM ADAPTABILITY DEMONSTRATED

The successful transition from IPC/CrPC to BNS/BNSS/BSA demonstrates an important property of the AI Lawyer Pro architecture: adaptability to legal framework changes. Because the system's legal knowledge is entirely stored in the vector database and knowledge graph (not baked into the LLM weights), updating the system to a new legal framework requires only updating the data pipeline and knowledge base, not retraining any AI models. This architectural advantage is a significant practical benefit for a system operating in a domain where laws evolve regularly.

Chapter 16: Experimental Phase – Page-Based Retrieval

16.1 EXPERIMENT MOTIVATION

During the development process, an alternative indexing strategy was explored: instead of chunking sections at the character level (1,000 characters with 200-character overlap), entire document pages were indexed as single units. This approach was motivated by the observation that legal sections sometimes span multiple chunks in the character-based system, potentially splitting a complex legal definition across two chunks and reducing retrieval precision.

16.2 PAGE-BASED APPROACH

In the page-based retrieval experiment, each page of the legal PDF was embedded as a single unit and stored in a separate FAISS-based test index. The potential advantages of this approach were:

- **Guaranteed completeness:** No legal definition could be split across page boundaries in the same way it might be split across character-based chunks
- **Simpler referencing:** The page number provides a natural, human-readable citation that could be shown to users
- **Better structural alignment:** Legal books are paginated by publishers with awareness of content boundaries

16.3 LIMITATIONS DISCOVERED

Testing revealed that page-based retrieval had critical limitations that made it unsuitable as the primary retrieval method:

- **Context window cost:** Each page contains significantly more text than a 1,000-character chunk. Passing multiple full pages to the LLM quickly fills the context window, increasing API costs and reducing answer focus
- **Reduced precision:** Pages may contain multiple sections, only one of which is relevant to the query. The LLM receives excessive irrelevant context alongside the relevant section
- **Scalability:** For a large legal corpus, page-based indexing produces fewer, larger index entries, reducing the granularity of semantic matching

16.4 DECISION

Based on this analysis, page-based retrieval was not adopted as the primary method. The experiment confirmed that character-based chunking with overlap strikes a better balance between context completeness and retrieval precision for large legal document corpora. Page-based indexing was retained only as a supplementary option for specific use cases involving smaller, more focused legal documents where page-level context is explicitly desired.

Chapter 17: Results and Evaluation

17.1 EVALUATION METHODOLOGY

The system was evaluated through a combination of qualitative analysis of response accuracy, quantitative comparison of retrieval configurations, and end-to-end pipeline testing with sample FIRs. A set of 50 test queries covering different aspects of BNS, BNSS, and BSA were used for systematic evaluation, including queries about offense classification, arrest procedure, bail eligibility, evidence requirements, and punishment ranges.

17.2 RETRIEVAL PRECISION IMPROVEMENT

Table 17.1 shows the measured improvement in retrieval precision at each development phase:

Configuration	Precision@5	Notes
Basic RAG (Vector Only)	P@5 = 0.62	Baseline system without reranking
RAG + Reranking	P@5 = 0.81	+30.6% improvement over baseline
Hybrid RAG (Vector + Graph)	P@5 = 0.87	+40.3% improvement over baseline
Hybrid RAG + Reranking	P@5 = 0.93	+50.0% improvement over baseline

Note: P@5 (Precision at 5) measures the fraction of the top 5 retrieved results that are genuinely relevant to the query, as assessed by manual evaluation.

17.3 HALLUCINATION RATE

The citation verification system measured the hallucination rate—the fraction of legal section citations in LLM responses that could not be verified against the ChromaDB metadata—across different configurations:

Configuration	Hallucination Rate
Standard LLM (No RAG)	~23% hallucination rate
Basic RAG	~8% hallucination rate
Hybrid RAG with Citation Verification	~2% hallucination rate (flagged and corrected)

The Hybrid RAG with citation verification reduced the hallucination rate by approximately 91% compared to a standard LLM, and by approximately 75% compared to basic RAG. The remaining 2% represents edge cases where the LLM synthesized information from multiple sections in a way that produced a citation that was technically present in the database but used in an incorrect context—a more subtle form of hallucination that requires human expert review.

17.4 SYSTEM PERFORMANCE

End-to-end query response time was measured for the complete pipeline:

- Vector retrieval (ChromaDB): < 100ms for top-20 candidates
- Cross-encoder reranking: ~800ms for 20 candidates
- Graph traversal (NetworkX): ~200ms for 3-hop neighborhood
- LLM generation (Groq API): ~1.5–3s depending on output length
- Citation verification: ~150ms
- Total end-to-end: ~3–4 seconds per complete FIR analysis

17.5 QUALITATIVE ASSESSMENT

Qualitative review of system responses demonstrated that AI Lawyer Pro correctly:

- Identified the applicable BNS/BNSS/BSA sections for 47 out of 50 test queries (94%)
- Provided accurate procedural guidance (arrest procedure, bail eligibility) for 45 out of 50 queries (90%)
- Generated syntactically valid JSON for visualization in 48 out of 50 cases (96%), with 2 cases requiring minor auto-correction
- Correctly flagged hallucinated citations in all cases where the LLM cited a non-existent section (100% detection rate)

Chapter 18: Future Enhancements

18.1 HYBRID SEARCH (BM25 + SEMANTIC)

While semantic search handles meaning-based queries excellently, it can struggle with exact Latin legal maxims (e.g., "actus reus", "mens rea") and specific article numbers where keyword precision is more important than semantic similarity. A future enhancement involves combining BM25 keyword search with the current semantic vector search in a hybrid retrieval approach, using Reciprocal Rank Fusion (RRF) to merge the results. This would give the system the benefits of both precision (BM25) and recall (semantic) in a unified retrieval pipeline.

18.2 ENHANCED GRAPH RAG WITH NEO4J

The current GraphRAG implementation uses NetworkX, an in-memory Python library. For production scale—handling hundreds of concurrent users and a knowledge graph extended to cover civil law, constitutional law, and judicial precedents—a dedicated graph database like Neo4j would be required. Neo4j provides persistent, optimized graph storage with a query language (Cypher) designed specifically for graph traversal, enabling much more complex multi-hop reasoning at scale.

18.3 JUDICIAL PRECEDENT INTEGRATION

The current system grounds its analysis in statutory law (BNS, BNSS, BSA) but does not incorporate judicial precedents—the body of case law from the Supreme Court of India and High Courts. Integrating judgments from Indian Kanoon or similar legal databases would transform the system from a statutory law analyzer into a comprehensive legal research assistant that can cite both applicable sections and relevant case precedents.

18.4 MULTI-LANGUAGE SUPPORT

Many FIRs in India are written in Hindi, Telugu, Tamil, or other regional languages. Extending the OCR system to support multilingual text recognition and adding

multilingual embedding models (e.g., LaBSE or multilingual-e5) would dramatically expand the system's practical applicability across India's linguistically diverse legal ecosystem.

18.5 FINE-TUNED LEGAL LLM

Currently, the system uses a general-purpose LLM (Llama-3.1-8b-instant) grounded in legal context through RAG. A future enhancement involves fine-tuning a smaller, specialized legal LLM on Indian legal text, combining domain adaptation with the RAG architecture. A fine-tuned model would better understand legal terminology, produce more precise legal reasoning, and require fewer tokens to generate accurate structured outputs.

18.6 UI ENHANCEMENTS

Future UI improvements include: answer highlighting that shows exactly which sentence in which legal section supports each claim; multi-document comparison for analyzing related FIRs; export functionality for generating formatted legal analysis reports in PDF; and a mobile-responsive design for access on smartphones, which are the primary internet-connected devices for many Indian legal professionals.

Chapter 19: Conclusion

This project successfully demonstrates the design and implementation of an advanced Legal AI system—AI Lawyer Pro—that overcomes the fundamental limitations of standard LLM applications in the legal domain. Through a disciplined, iterative development process, the system evolved from a basic chunk-based retrieval pipeline into a sophisticated Hybrid RAG architecture combining semantic vector search with graph-based relational reasoning.

The key technical achievements of this project are:

28. A reliable, metadata-enriched data pipeline sourced directly from Official Bare Act PDFs of BNS, BNSS, and BSA—ensuring the knowledge base is legally authoritative and current.
29. A semantic retrieval system using SentenceTransformers and ChromaDB that enables meaning-based legal search beyond keyword matching.
30. A precision reranking layer using a Cross-Encoder (ms-marco-MiniLM-L-6-v2) that elevates the most precisely relevant legal sections to the top of the retrieval results.
31. A citation verification system that actively validates LLM outputs against the database, reducing hallucination by approximately 91% compared to ungrounded LLMs.
32. A GraphRAG implementation using NetworkX with 1,140+ nodes and 1,860+ edges that enables multi-hop legal reasoning across BNS/BNSS/BSA relationships.
33. An advanced prompt engineering strategy (zero temperature, relevancy filtering, few-shot JSON schema enforcement) that ensures deterministic, structured, and visualization-ready outputs.
34. An interactive Streamlit frontend with a Plotly visualization dashboard providing case timelines, risk scores, and section impact analysis.
35. A dual-pipeline OCR system using Tesseract that handles both digital and scanned legal PDF documents.
36. A microservices deployment architecture (FastAPI + Streamlit + Ngrok) enabling scalable, maintainable, and publicly demonstrable deployment.

The successful transition of the system from IPC/CrPC to BNS/BNSS/BSA demonstrates the adaptability of the architecture to legal framework changes—a critical property for a system operating in a dynamic legal environment. The project represents not merely a technical exercise but a practical contribution to the emerging field of Indian Legal AI, addressing a real and pressing need created by the 2023 criminal law reforms.

AI Lawyer Pro stands as evidence that combining careful data engineering, multi-modal retrieval architecture, robust output validation, and thoughtful user interface design can produce a Legal AI system that is not just impressive in demonstration, but reliable, explainable, and genuinely useful in the challenging domain of legal document analysis.

References

37. P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," arXiv:2005.11401, 2020.
38. B. Peng, X. Liu, and Y. Chen, "Graph Retrieval-Augmented Generation: A Survey," arXiv, 2024.
39. H. Han, J. Wang, and Z. Li, "GraphRAG: Enhancing Retrieval-Augmented Generation with Graph Structures," arXiv, 2024.
40. NVIDIA Developer Blog, "What is Retrieval-Augmented Generation (RAG)?," 2025.
41. A. Sinha, "Understanding Vector Databases in RAG Systems," Medium, 2025.
42. Hybrid RAG Systems Research Paper, "Hybrid Retrieval-Augmented Generation Systems with Embedding Vector Databases," ResearchGate, 2025.
43. Z. Li, et al., "Applications of Retrieval-Augmented Generation in Knowledge Systems," ScienceDirect, 2025.

Appendix A: System Implementation Details

A.1 DATA ENGINEERING PIPELINE – DETAILED IMPLEMENTATION

This appendix provides the detailed implementation notes for each phase of the AI Lawyer Pro system. The descriptions below elaborate on the key code-level decisions made during development.

A.1.1 PDF TEXT EXTRACTION AND CLEANING

The first step in the data pipeline was extracting raw text from the Official Bare Act PDFs using PyPDF2. The extraction produced raw strings that required several cleaning steps before use:

- Header and footer removal: Each page of a legal act PDF repeats the act name and page number in the header/footer. A pattern-matching step identified and removed these repetitions from the extracted text.
- Hyphenation handling: Words split across line breaks by hyphens (e.g., 'non- bailable') were re-joined to ensure semantic integrity.
- Unicode normalization: Legal texts occasionally contain special Unicode characters from older typesetting. A normalization step converted these to standard ASCII equivalents where appropriate.
- Whitespace normalization: Multiple consecutive whitespace characters, tabs, and unusual line break patterns were normalized to single spaces.

After cleaning, the text was structured into a list of (section_number, section_title, section_content) tuples using the Regex-based section parser. Each tuple was validated to ensure completeness before metadata enrichment.

A.1.2 METADATA ENRICHMENT PROCESS

Metadata enrichment was a semi-automated process. The procedural metadata (bailable/non-bailable, cognizable/non-cognizable) was extracted from the Schedule tables appended to the BNSS, which contain explicit classifications for each offense category

under BNS. A cross-referencing algorithm matched BNS section numbers to their BNSS Schedule entries, populating the classification metadata for each section.

For sections where the Schedule did not provide an explicit classification (e.g., procedural sections of BNSS, evidentiary sections of BSA), the metadata fields were left as null and filtered from classification-dependent queries at retrieval time.

A.2 CHROMADB COLLECTION STRUCTURE

The ChromaDB vector database was organized into three separate collections, one per law:

Collection Name	Contents
bns_collection	Bharatiya Nyaya Sanhita – 358 sections
bnss_collection	Bharatiya Nagarik Suraksha Sanhita – 531 sections
bsa_collection	Bharatiya Sakshya Adhiniyam – 170 sections
Total	1,059 primary sections (plus sub-sections = ~3,200 chunks after chunking)

Using separate collections per law enables targeted retrieval: when a query is specifically about arrest procedure, only the BNSS collection needs to be searched, significantly improving retrieval speed and precision.

A.3 CROSS-ENCODER RERANKING – TECHNICAL DETAILS

The Cross-Encoder model (ms-marco-MiniLM-L-6-v2) was loaded using the sentence-transformers library's CrossEncoder class. The reranking process:

44. Receives a list of (query, candidate_chunk) pairs
45. Passes each pair through the Cross-Encoder's full attention transformer
46. Receives a relevance score (float) for each pair
47. Sorts candidates by descending relevance score
48. Returns the top-5 candidates for LLM context assembly

The Cross-Encoder inference is run on CPU for compatibility with diverse deployment environments. GPU acceleration can be enabled by setting the device parameter to 'cuda' when a CUDA-compatible GPU is available, reducing reranking time from ~800ms to ~80ms.

A.4 GRAPH RAG IMPLEMENTATION – NETWORKX DETAILS

The knowledge graph was serialized as a JSON file (legal_graph.json) that encodes all nodes, edges, and their associated metadata. The JSON format was chosen over a binary format (pickle) for portability and human readability, enabling manual inspection and debugging of the graph structure.

At application startup, the graph is loaded from legal_graph.json into a NetworkX DiGraph object. The graph traversal algorithm for query time operates as follows:

49. Seed node identification: The section numbers from the top-3 vector-retrieved chunks are used as seed nodes
50. Breadth-first expansion: Starting from each seed node, the algorithm performs a breadth-first traversal up to depth=2 (two hops), collecting all encountered nodes
51. Deduplication: Nodes already in the vector retrieval results are excluded from the graph results to avoid content duplication in the LLM context
52. Content extraction: The full content of all collected graph nodes is retrieved from the node attribute store
53. Context assembly: Graph content is appended to vector content with a clear section header indicating its source ('Retrieved via graph traversal')

The depth=2 traversal parameter was chosen after empirical testing. A depth=1 traversal (immediate neighbors only) missed important cross-chapter references. A depth=3 traversal collected too many tangentially related sections, increasing context length without proportional improvement in answer quality.

Appendix B: Testing and Sample Query Analysis

B.1 TEST QUERY CATEGORIES

The system was evaluated against five categories of test queries, each designed to test different aspects of the system's capabilities:

Category	Description
Category 1: Offense Classification	Queries about what specific acts constitute under BNS (murder, theft, assault, etc.)
Category 2: Procedural Queries	Queries about arrest procedure, bail eligibility, trial process under BNSS
Category 3: Evidence Queries	Queries about admissibility of evidence, digital evidence, testimony rules under BSA
Category 4: Multi-Hop Queries	Complex queries requiring traversal across BNS + BNSS or BNSS + BSA sections
Category 5: FIR Analysis	Complete FIR document analysis requiring identification of all applicable sections

B.2 SAMPLE QUERY AND RESPONSE ANALYSIS

The following examples illustrate the system's response quality across different query categories:

B.2.1 SAMPLE QUERY 1: OFFENSE CLASSIFICATION

Query: 'What is the punishment for murder under the new Indian law?'

System Response Summary:

Applicable Section: BNS Section 101 (Murder)

Definition: Culpable homicide is murder if the act is done with the intention of causing death, or causing bodily injury likely to cause death.

Punishment: Death, or imprisonment for life, and a fine.

Classification: Cognizable, Non-Bailable, Triable by Court of Session

Verified Citations: BNS 101 [VERIFIED], BNSS Schedule I [VERIFIED]

B.2.2 SAMPLE QUERY 2: MULTI-HOP PROCEDURAL QUERY

Query: 'If a person is arrested for robbery, can they get bail? What is the procedure?'

System Response Summary (Multi-Hop Chain):

Step 1: BNS Section 309 (Robbery) - Offense identified and classified

Step 2: BNSS Schedule I - Classification retrieved: Non-Bailable, Cognizable

Step 3: BNSS Section 480-483 (Bail provisions) - Bail conditions retrieved

Step 4: BNSS Section 35 (Arrest procedure for cognizable offense) - Retrieved

Conclusion: Bail is not a matter of right but at Court's discretion.

Procedure: Police may arrest without warrant (cognizable). Bail applied at Magistrate.

Verified Citations: BNS 309 [VERIFIED], BNSS 480 [VERIFIED], BNSS 35 [VERIFIED]

This example demonstrates the GraphRAG capability: the query required traversing from the offense section (BNS 309) through the classification schedule (BNSS Schedule I) to the bail provisions (BNSS 480-483) and arrest procedure (BNSS 35). A pure vector search would have retrieved only sections semantically similar to 'robbery bail procedure', potentially missing the precise procedural chain.

B.3 EDGE CASES TESTED

- Query about a section number that does not exist in BNS/BNSS/BSA: System correctly indicated no matching section and suggested the closest valid section.
- Query in informal language ('what happens if someone beats me up?'): System correctly identified the applicable BNS sections for assault and grievous hurt.
- Query mixing old and new law terminology ('IPC Section 302'): System recognized the old section reference and correctly mapped it to the equivalent BNS section.

- Scanned FIR PDF with poor scan quality: OCR pipeline successfully extracted text with minor character recognition errors that did not affect section identification.
- FIR containing no criminal offense (civil dispute described as criminal): System correctly identified that no BNS criminal sections applied and informed the user.

Appendix C: Technology Comparison and Selection Rationale

C.1 EMBEDDING MODEL COMPARISON

Several embedding models were evaluated before selecting all-MiniLM-L6-v2:

Model	Status	Rationale
all-MiniLM-L6-v2	Selected	Fast, 384-dim, good quality, local execution
all-mpnet-base-v2	Considered	Better quality, 768-dim, 2x slower, higher memory
OpenAI text-embedding-3-small	Rejected	External API, cost per request, privacy concern
legal-bert-base-uncased	Considered	Legal domain fine-tuned but lower general performance
multilingual-e5-large	Future	For multi-language support in future enhancement

C.2 VECTOR DATABASE COMPARISON

Database	Status	Rationale
ChromaDB	Selected	Lightweight, local, persistent, metadata filtering, easy Python API
FAISS	Used in experiment	Fastest retrieval, but no persistence and no metadata filtering
Weaviate	Rejected	Requires Docker, external server, overkill for project scope
Pinecone	Rejected	Cloud-based, privacy concern for legal documents, cost
Qdrant	Considered	Good alternative, but ChromaDB simpler for this use case

C.3 LLM PROVIDER COMPARISON

Provider	Status	Rationale
Groq + Llama-3.1-8b-instant	Primary (Selected)	Fastest inference, free tier, good legal reasoning
Ollama (local)	Fallback	Offline capability, privacy, no API dependency
OpenAI GPT-4	Rejected	High cost, external data transfer, overkill
Anthropic Claude	Considered	Excellent quality, but API cost for sustained use
Cohere Command	Considered	Good RAG integration, but less open-source community

C.4 GRAPH LIBRARY COMPARISON

Library/DB	Assessment
NetworkX (Selected)	In-memory Python graph library. Perfect for project scale. No external server. Serializable to JSON. Comprehensive algorithm library.
Neo4j (Future)	Production-grade graph database. Cypher query language. Persistent storage. Better for large-scale, multi-user production deployment.
ArangoDB (Considered)	Multi-model DB supporting graphs. Good but adds infrastructure complexity beyond project scope.
GraphDB (Rejected)	RDF-based triple store. Overkill for legal section relationships. Steep learning curve.

Appendix D: Project Evolution Timeline

D.1 DEVELOPMENT MILESTONES

The following table chronicles the complete development history of AI Lawyer Pro, showing how the system evolved from a basic prototype to a sophisticated Hybrid RAG system:

Phase	Milestone	Description
Phase 1	Data Source Evaluation	Evaluated CSV datasets; identified critical metadata gaps; pivoted to Official Bare Act PDFs
Phase 2	PDF Pipeline Development	Built PyPDF2 extraction + Regex section parser + metadata enrichment pipeline
Phase 3	Vector DB Setup	Deployed ChromaDB; indexed BNS/BNSS/BSA chunks with SentenceTransformers embeddings
Phase 4	Basic RAG	First working RAG pipeline: query → embedding → ChromaDB → LLM; identified ranking issues
Phase 5	Reranking Layer	Added Cross-Encoder (ms-marco-MiniLM-L-6-v2); 30% precision improvement
Phase 6	Hallucination Prevention	Built Regex citation extractor + ChromaDB cross-checker; flagging system operational
Phase 7	Frontend v1	Basic Streamlit UI with authentication and ChatGPT-style interface
Phase 8	Microservices Architecture	Separated backend (FastAPI/Uvicorn:8000) from frontend (Streamlit:8501); REST API comm
Phase 9	Deployment	venv isolation + Ngrok tunneling; system accessible for live demo and testing
Phase 10	Page-Based Experiment	Tested FAISS page-indexed retrieval; identified limitations; confirmed chunk strategy

Phase	Milestone	Description
Phase 11	Legal Framework Update	Transitioned from IPC/CrPC to BNS/BNSS/BSA; rebuilt entire knowledge base
Phase 12	GraphRAG Implementation	Built NetworkX directed multigraph (1140 nodes, 1860 edges); integrated with RAG pipeline
Phase 13	Hybrid RAG	Fused vector + graph retrieval; parallel execution + results merger + unified reranking
Phase 14	Advanced Prompt Engineering	Zero temperature + relevancy filtering + few-shot JSON schema enforcement
Phase 15	Visualization Dashboard	Plotly scatter (timeline) + gauge (risk) + bar (section impact) charts integrated
Phase 16	Error Handling	Schema validation, auto-correction logic, safe fallbacks for all edge cases
Phase 17	OCR Integration	Dual-pipeline: standard extraction + Tesseract OCR fallback for scanned PDFs
Phase 18	System Testing	50-query evaluation suite; 94% section identification accuracy; 2% hallucination rate

D.2 KEY ARCHITECTURAL DECISIONS AND THEIR IMPACT

Looking back at the full development journey, three architectural decisions had the most significant impact on the final system quality:

Decision 1: Official Bare Act PDFs over CSV datasets

This decision, made in Phase 1, established the principle of grounding the system in authenticated legal sources. While it significantly increased the data engineering complexity (requiring custom Regex parsers and metadata enrichment), it ensured the legal knowledge base was legally authoritative, complete, and aligned with the official BNS/BNSS/BSA text. No subsequent accuracy issues were traceable to incorrect source data.

Decision 2: Hybrid RAG over pure vector search

The addition of GraphRAG (Phase 12) was the most technically ambitious enhancement. The 1,140-node, 1,860-edge knowledge graph required significant upfront construction effort but yielded qualitative improvements in multi-hop legal reasoning that vector search alone could not provide. The graph enabled the system to answer complex procedural queries (e.g., 'arrest → classification → bail procedure' chains) that would have required multiple separate queries to a pure vector-based system.

Decision 3: Citation verification layer

The hallucination prevention system (Phase 6) was a distinctive engineering choice that differentiated AI Lawyer Pro from a simple LLM wrapper. By actively validating outputs against the database, the system transformed from a best-effort AI assistant into one with explicit output quality guarantees. This safety layer is essential for a legal AI system and demonstrates the engineering maturity required for responsible AI deployment in high-stakes domains.

Appendix E: System Requirements and Setup

E.1 HARDWARE REQUIREMENTS

Component	Specification
Minimum RAM	8 GB (for ChromaDB + SentenceTransformers in CPU mode)
Recommended RAM	16 GB (for comfortable multi-model concurrent operation)
CPU	Any modern multi-core CPU (4+ cores recommended for concurrent backend + frontend)
GPU	Optional (CUDA-compatible GPU accelerates Cross-Encoder reranking from ~800ms to ~80ms)
Storage	Minimum 5 GB (ChromaDB index + graph JSON + model weights)
Network	Internet connection required for Groq API; Ollama enables offline operation

E.2 SOFTWARE DEPENDENCIES

Library	Purpose
Python	3.10 or higher
PyPDF2	PDF text extraction
sentence-transformers	Embedding model (all-MiniLM-L6-v2)
chromadb	Vector database
networkx	Knowledge graph
fastapi	Backend REST API framework
uvicorn	ASGI server for FastAPI
streamlit	Frontend web application
plotly	Interactive visualizations
groq	Groq API client for Llama inference
ollama	Local LLM execution client
pdfplumber	Enhanced PDF text extraction
pytesseract	Python wrapper for Tesseract OCR

Library	Purpose
pdf2image	PDF to image conversion for OCR pipeline
python-dotenv	Environment variable management
requests	Frontend-to-backend REST API communication
	Figure 14.1: Deployment Architecture - Uvicorn + Streamlit + Ngrok <i>Figure 14.1: Deployment Architecture – Uvicorn + Streamlit + Ngrok</i>
torch	Deep learning backend for SentenceTransformers

E.3 SETUP INSTRUCTIONS

54. Clone the project repository to a local directory
55. Create a Python virtual environment: `python -m venv venv`
56. Activate the virtual environment: `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
57. Install all dependencies: `pip install -r requirements.txt`
58. Install Tesseract OCR engine (system-level): `sudo apt-get install tesseract-ocr` (Linux) or `brew install tesseract` (Mac)
59. Install poppler for PDF-to-image conversion: `sudo apt-get install poppler-utils` (Linux)
60. Create a .env file with `GROQ_API_KEY=your_api_key_here`
61. Run the data pipeline to build the ChromaDB index: `python data_pipeline.py`
62. Build the knowledge graph: `python build_graph.py`
63. Start the backend: `uvicorn main:app --host 0.0.0.0 --port 8000`

64. Start the frontend (new terminal): streamlit run app.py --server.port 8501
65. For public demo, start Ngrok: ngrok http 8501

E.4 PROJECT DIRECTORY STRUCTURE

The project repository is organized as follows to maintain clean separation of concerns:

```
ai-lawyer-pro/  
  backend/  
    main.py          -- FastAPI application entry point  
    rag_pipeline.py  -- Core RAG + reranking logic  
    graph_rag.py     -- NetworkX graph traversal  
    hallucination_check.py -- Citation verification system  
    ocr_pipeline.py  -- Dual-pipeline OCR handler  
    legal_graph.json -- Serialized knowledge graph  
  data/  
    data_pipeline.py -- PDF extraction + chunking + ChromaDB indexing  
    build_graph.py   -- Knowledge graph construction script  
    bare_acts/       -- Official Bare Act PDFs (BNS, BNSS, BSA)  
    master_legal_v1.json -- Structured legal knowledge base  
  frontend/  
    app.py           -- Streamlit frontend application  
    auth.py          -- Authentication system  
    visualizations.py -- Plotly chart generators  
  chroma_db/         -- ChromaDB persistent storage (auto-created)  
  .env               -- Environment variables (not in version control)  
  requirements.txt    -- Python dependencies  
  README.md          -- Setup and usage documentation
```

CLICK HERE FOR : [source code](#)

E.5 API ENDPOINTS

The FastAPI backend exposes the following REST API endpoints:

Endpoint	Name	Description
POST /analyze	FIR Analysis	Accepts multipart form data with FIR PDF file; returns complete structured JSON analysis with sections, timeline, risk score
POST /query	Legal Query	Accepts JSON with query string; returns RAG-generated legal answer with verified citations
GET /health	Health Check	Returns system health status and ChromaDB connectivity status
GET /stats	System Stats	Returns ChromaDB collection sizes and graph node/edge counts

All POST endpoints accept JSON bodies or multipart form data as appropriate, and return structured JSON responses. Error responses follow RFC 7807 Problem Details format, providing clear error messages for client-side handling. All endpoints include request validation through FastAPI's Pydantic schema integration, ensuring type safety and automatic documentation generation via the OpenAPI spec available at /docs.

E.6 SECURITY CONSIDERATIONS

The following security considerations were implemented in the system design:

- All uploaded legal documents are processed in memory and not persisted to disk beyond the duration of the analysis request, protecting the confidentiality of sensitive FIR content.
- The authentication system uses session-state-based access control implemented through Streamlit's session state mechanism with user credentials stored in a secured JSON store.
- API keys for Groq are stored exclusively in the .env file and never embedded in source code or transmitted to the frontend.
- Ngrok tunneling uses HTTPS encryption for all traffic in transit, preventing interception of uploaded legal documents during demo sessions.

- The microservices architecture provides an additional security boundary: the frontend has no direct access to the ChromaDB database or graph files; all data access is mediated through the FastAPI backend API.

Appendix F: Comparison with Related Work

F.1 AI LAWYER PRO VS. EXISTING LEGAL AI SYSTEMS

To contextualize the contributions of AI Lawyer Pro, this appendix compares it with existing legal AI systems and research prototypes:

System	Approach	Limitation vs. AI Lawyer Pro
ROSS Intelligence	Vector search over legal documents	No GraphRAG; US law focus; no BNS/BNSS/BSA
LexisNexis AI	Hybrid keyword + semantic search	Proprietary, not open; no Indian law; no hallucination check
Indian Kanoon Search	Full-text keyword search over Indian cases	No AI generation; no RAG; only case law, not statutes
Standard GPT-4 Legal Chatbot	Parametric LLM knowledge only	High hallucination; no retrieval; outdated knowledge
Basic RAG Legal System	Vector RAG without graph	No multi-hop reasoning; no GraphRAG; no citation verification
AI Lawyer Pro (This Project)	Hybrid Vector + GraphRAG + Citation Verification	BNS/BNSS/BSA specific; anti-hallucination; open architecture

F.2 KEY DIFFERENTIATORS

AI Lawyer Pro is distinguished from existing systems by the combination of five features that no single existing system provides simultaneously:

66. Grounded specifically in the updated Indian criminal law framework (BNS, BNSS, BSA) rather than older IPC/CrPC
67. Hybrid Retrieval combining semantic vector search with structured GraphRAG for multi-hop legal reasoning
68. Active hallucination prevention through post-generation citation verification against the knowledge base

- 69. Structured visualization output (timeline, risk score, section impact) rendered directly from LLM JSON output
- 70. Fully open architecture built on open-source components (no proprietary APIs for core functionality when using Ollama)

--- End of Report ---

LLM LAWYER: A RAG-BASED LEGAL REASONING AND ASSISTANCE
SYSTEM USING LLMS

VIT-AP University, Amaravati, Andhra Pradesh, India

Vulli Mohan Durga Prakash (22BCE9657) | Alur Manideep (22BCE20036)

Guide: Dr. Jonnadula Harikiran, Associate Professor